Journal of Network and Computer Applications 210 (2023) 103545



Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca





HELPHED: Hybrid Ensemble Learning PHishing Email Detection

Panagiotis Bountakas *, Christos Xenakis

Department of Digital Systems, University of Piraeus, 80 Karaoli & Dimitriou, Piraeus, 18534, Attica, Greece

ARTICLE INFO

Keywords:
Phishing email detection
Machine Learning
Ensemble Learning
Hybrid features
Natural Language Processing

ABSTRACT

Phishing email attack is a dominant cyber-criminal strategy for decades. Despite its longevity, it has evolved during the COVID-19 pandemic, indicating that adversaries exploit critical situations to lure victims. Plenty of detectors have been proposed over the years, which mainly focus on the contents or the textual information of emails; however, to cope with the evolution of phishing emails more sophisticated approaches should be introduced that will exploit all the emails' traits to enhance the detection capability of Machine Learning/Deep Learning classifiers. To tackle the limitations of existing works, this paper proposes a phishing email detection methodology, named HELPHED that focuses on the detection of phishing emails by combining Ensemble Learning methods with hybrid features. The hybrid features provide an accurate representation of emails by fusing their content and textual traits. We propose two methods of HELPHED, the first one employs the Stacking Ensemble Learning method, while the second method utilizes the Soft Voting Ensemble Learning. Both methods deploy two different Machine Learning algorithms to handle the hybrid features separately, yet in parallel, minimizing the features' complexity and improving the model's performance. A thorough evaluation analysis is carried out considering innovative guidelines that aim to prevent partial and misleading results. Experimental tests verified that the combination of hybrid features with Ensemble Learning, overall, accomplishes better detection performance than when employing only content-based or text-based features. Numerical results on a rich imbalanced dataset (i.e., 32,051 benign and 3,460 phishing email samples) that considers the evolution of phishing emails show that Soft Voting Ensemble Learning outperforms other prominent Machine Learning/Deep Learning algorithms and existing works yielding F1-score equal to 0.9942.

1. Introduction

The wide expansion of emails render them a priceless tool and at the same time a growing and hazardous threat. Phishing email campaigns are the most pernicious cyber-criminal strategy to perform malicious actions that often act as a preliminary stage for conducting large-scale and sophisticated attacks (e.g., APTs, Exploit Kits) (Bountakas et al., 2022). Despite the longevity of phishing email attacks, traditional email filtering techniques have been inefficient in effectively confronting phishing email attacks. The latest phishing reports highlight that in 2020 75% of organizations faced a phishing attack, as well as that 96% of phishing attacks were performed via emails (Anon., 2021b), while the email threats rose by 64% compared to 2019 (Anon., 2021c). Moreover, phishing attacks account for 90% of data breaches provoking high financial losses of the order of $3.86 million (Anon., 2021a). The expansion of COVID-19 over the last couple of years, cultivated a breeding ground for email attacks since between February 2020 and March 2020 phishing emails were raised by 667% (Anon., 2020b). More specifically, cyber-criminals exploit people's desperation (regarding the pandemic, vaccination, etc.) using widespread awareness of the COVID-19 to deceive them into clicking on malicious links or attachments. Since the onset of the pandemic, employees are clicking three times as many malicious emails as they had before. The phishing attacks related to COVID-19 were highly successful as the attackers imitated ministries of health, health centers, governments, or important organizations of a relevant country to be confused as reliable sources (Anon., 2020d). Furthermore, in the last few years, the involvement of Machine Learning (ML) in the generation process of phishing attacks has led to more sophisticated and identical to benign phishing emails that can evade current detection solutions (Yamin et al., 2021).

Phishing emails have drawn the focus of academia for more than a decade, where over the years scientists have proposed numerous detection methods, mainly utilizing ML methods, to thwart phishing email attacks. On the other hand, several recent studies (e.g., Das et al., 2020; El Aassal et al., 2020; Gangavarapu et al., 2020) have identified various gaps and limitations in the literature, such as (i) a narrow set of classifiers is usually considered, (ii) balanced datasets are often

* Corresponding author.
E-mail addresses: bountakas@unipi.gr (P. Bountakas), xenakis@unipi.gr (C. Xenakis).

https://doi.org/10.1016/j.jnca.2022.103545
Received 14 June 2022; Received in revised form 21 October 2022; Accepted 8 November 2022
Available online 21 November 2022
1084-8045/© 2022 Elsevier Ltd. All rights reserved.

employed, even though phishing detection is an imbalanced classification problem, (iii) inadequate evaluation metrics, (iv) dataset issues (i.e., limited available data sources for researchers), and (v) obsolete phishing emails. The aforementioned pitfalls affect the applicability of the existing works in real-life conditions. Furthermore, the previous ML works mostly focus either on the email's contents (i.e., attributes of an email that represent its structure, like headers, URLs, syntax, attachments, etc.) or on the email's body text (applying Natural Language Processing and text analysis) to extract representative features. To the best of our knowledge, there are no recent works that combine the emails' contents and body text to provide a thorough representation of emails that can be processed by ML algorithms. Additionally, Ensemble Learning (e.g., the combination of two or more ML algorithms) has progressed the results in several fields of research, such as phishing web page detection (Li et al., 2019) and fraud detection (Haider et al., 2018); however, it has not been comprehensively examined in the field of phishing email detection yet. Based on the aforementioned evidence, the motivation of this research arises from the evolution of phishing email attacks, which has rendered many of the existing detection solutions obsolete as well as from the identified limitations of the literature, which highlight the fact that phishing email attacks pose a viable threat and their detection is still a work in progress. Therefore, new robust solutions are required to curb the expansion of phishing email attacks.

Towards this direction, trying to detect a phishing email campaign by delving into particular features (e.g., contents or body text) can be considered as "a day late and a dollar short", since modern sophisticated phishing emails are identical to benign, and hence their detection is more challenging than it was the early years. We argue that hybrid features (i.e., the combination of content and body text features) include a wealth of information that provides a thorough representation of phishing emails capturing even the slightest variations. Even though the basic idea is simple, a major challenge is how to effectively process the hybrid features to exploit all the information they can offer. Conventional learning methods (e.g., traditional ML algorithms) are not able to process the hybrid features effectively since, even though the structure of the content-based and text-based features heavily differs, the traditional ML algorithms treat them similarly. On the contrary, Ensemble Learning can exploit all the information of the hybrid features by implementing two different algorithms (i.e., the algorithm that attains the best classification results with the content-based features and the algorithm that attains the best results with the text-based features) to process the hybrid features separately; hence improving the phishing email detection performance. Regarding the observation mentioned above, this work aspires to answer the following research question: *Can the combination of Ensemble Learning with hybrid features, overall, enhance the phishing email detection performance? (RQ)*

In this paper, we propose an innovative data-driven *Ensemble Learning* methodology, named HELPHED (Hybrid Ensemble Learning PHishing Email Detection) that implements two different Ensemble Learning methods and highly informative *hybrid features* to efficiently detect phishing emails. HELPHED aims to enhance the prediction results in the phishing email detection domain by classifying phishing emails accurately, quickly, and cost-effectively, exploiting all the information that emails' contain. Using hybrid features, namely features obtained both from the message body text (i.e., the email's message that is shown to the recipient) and the contents of emails (i.e., header fields, most used words in phishing emails, attachments, etc.), we achieve to convert the email into a compact, yet thorough representation that can be efficiently interpreted by the Ensemble Learning methods to acquire high detection performance. The methodology is comprised of 6 stages. First, the *Email Parsing* stage divides the email contents and stores them in arrays to ease their processing. Next, the *Content Feature Extraction* stage extracts 22 features from the emails' contents, which are divided in four categories: *body*, *syntactic*, *header*, and *URL*. In parallel, the *Pre-processing* stage prepares the email's body text for the *Textual Feature Extraction* stage, which employs the Word2Vec method to acquire text-based features. Later, the *Feature Selection* stage deploys the Mutual Information method to select the most informative features of the hybrid feature sets. Last but not least, the binary classification of the emails between benign and phishing is performed in the *Ensemble Classification* stage, where two methods are proposed, *Method 1 - Stacking Ensemble Learning* and *Method 2 - Soft Voting Ensemble Learning*. These methods are comprised of two ML algorithms to process the hybrid features separately and in parallel (i.e., the Decision Tree ML algorithm processes the content-based features, while k-Nearest Neighbors process the text-based features).

Emphasis was put on the evaluation of the proposed methodology to abide by the proposed guidelines for the assessment of ML methods in phishing email detection to avoid biased and misleading results. Particularly, a large imbalanced dataset, which includes various data sources and newer phishing emails (2015–2021) was deployed to assess the proposed methodology. Numerical results (based on numerous evaluation metrics appropriate for class imbalance) indicate that the combination of hybrid features and Ensemble Learning improves the detection performance in the phishing email detection domain. Both *Method 1* and *Method 2* attain comparable results, however, *Method 2* accomplished the best F1-score (0.9942), MCC (0.967), accuracy (0.9943), as well as low training time (0.0313 s).

In summary, the contributions of this paper lie in the following aspects:

- We introduce the concept of hybrid email features that offer an accurate, yet thorough representation of benign and phishing emails.
- We propose an Ensemble Learning phishing email detection methodology that is designed to effectively process the hybrid features employing two methods (i.e., Stacking and Soft Voting).
- We present five innovative guidelines for the objective assessment of the proposed methodology, which are usually neglected by existing works leading to biased and inaccurate results.
- We share a rich email dataset that is comprised of an imbalanced ratio of real-life phishing and benign emails as well as it considers the evolution of phishing email attacks.
- We prove that the combination of Ensemble Learning with hybrid features, overall, improves the detection performance of phishing emails.

The remainder of the paper is structured as follows: In Section 2, it is presented the related research work, including neighboring approaches towards email phishing detection, and discussed their limitations. Section 3 elaborates on the proposed methodology. In Section 4, the assessment is described along with innovative evaluation guidelines, and the outcomes of the experiments are presented and discussed. Lastly, Section 5 outlines the critical points of the paper.

2. Related work

The related work section elaborates on existing works that are based on ML phishing email detection. Given that emails cannot be directly processed by ML algorithms, feature engineering is imperative to convert an email into a compact representation that can be interpreted by an ML algorithm. Existing works mainly focus on content or text-based features to distinguish benign from phishing emails. Thus, we grouped the related works in two categories based on the extracted features: (a) detection of phishing emails using content-based features and (b) detection of phishing emails using text-based features. Later, we discuss the application of Ensemble Classification in the phishing email detection field and the section concludes with the limitations of the literature and how these have been addressed by this work.

2.1. Detection of phishing emails using content-based features

Ma et al. (2009) were among the first that introduced the concept of hybrid feature extraction for phishing email detection. The hybrid feature set included content, orthographic, and a combination of content and orthographic features. The proposed solution accomplished high detection accuracy with a decision tree (C4.5) classifier on a highly imbalanced dataset (only 7% of emails were phishing). However, this work evaluated only on accuracy that is an inappropriate metric for imbalanced data and the results might be biased towards the majority class (Bhowan et al., 2013). Hamid and Abawajy (2011) also presented a hybrid approach that combines content-based and behavior-based features. The authors tested their approach using a Bayesian Network classifier on old phishing emails (2004–2007) and reached 92%–96% detection accuracy. This approach heavily relied on domain and URL features, hence it can be easily evaded by newer more sophisticated phishing emails. Both Ma et al. (2009) and Hamid and Abawajy (2011) were proposed more than a decade ago, hence their efficacy on newer phishing emails is disputable. In this work, we also embrace the concept of hybrid features, but instead of the integration of behavior-based and content-based features, we combined content-based with text-based features.

Moradpoor et al. (2017) proposed an approach that is comprised by six components and deploys an Artificial Neural Network (ANN) to detect phishing emails. The evaluation performed on public email datasets and accomplished 92.2% detection accuracy. This work has several limitation, as the features extracted from the emails (four in total) are not sufficient, which results in lower detection accuracy than other works in the literature. We addressed this issue and increased the accuracy by extracting more representative features and instead of using the average value of word vectors as a feature, we exploited all the word vectors from the emails' text by using them as input in the classifier. In Akinyelu and Adewumi (2014), the authors extracted in total 15 features from the email's contents and employed a Random Forest classifier to perform the classification based on 10-fold cross validation and achieved 99.7% classification accuracy and 98.45% F1-score. This work accomplished comparing results; however, the authors deployed a private and imbalanced dataset that contains a small number of emails (1800 benign and 200 phishing), as well as they did not deploy appropriate evaluation metrics for imbalanced data.

Smadi et al. (2015) proved that the pre-processing phase can enhance the efficacy of the classifier. Particularly, the authors divided the feature extraction process in three phases and in each phase extracted different features mainly from the URLs. The paper concludes that the Random Forest classifier accomplished the best accuracy (98.87%). Marchal et al. (2014) focused on the URLs that included in the emails and defined the term intra-URL relatedness which practically investigates the relationship between the words that comprising the various fields of a URL. The evaluation showed that the Random Forest classifier achieved the higher detection accuracy (94.91%). Both Smadi et al. (2015) and Marchal et al. (2014) have common limitations. For instance, they focused on emails' URLs; hence, in case that there are no URLs (e.g., a phishing email that delivers a malicious file), these approaches will be ineffective. In Smadi et al. (2015) the authors used publicly available datasets, but they did not mention the emails' creation date, while in Marchal et al. (2014) the authors tested their approach only on URLs, thus it could not be considered as a generic phishing email solution. Furthermore, there are also other content-based works in the literature (e.g., Fette et al., 2007; Chandrasekaran et al., 2006; Abu-Nimeh et al., 2007); however, they can be considered outdated, since they were proposed the previous decade (before 2010) and it is not sure if they are able to follow the advancement of phishing emails (i.e., a re-evaluation is needed with newer phishing and benign emails).

2.2. Detection of phishing emails using text-based features

An interesting observation is that the older works (before 2015) on phishing email detection primarily focus on content-based features, while the most recent works focus on textual features. This observation can be attributed to the recent evolution of Artificial Intelligence (AI) and NLP methods, which have contributed to the robust detection of phishing emails by using only the textual information that appears in them.

A recently published work proposed in Alhogail and Alsabih (2021), where the authors deployed the TF-IDF method to create the text-based feature set and a Graph Convolution Network to classify the emails. The experiments on 3685 phishing and 4894 benign emails (Radev, 2008), using 3-fold cross-validation concluded to 98.2% accuracy. The drawbacks of Alhogail and Alsabih (2021) is that the number of phishing emails is much less than the benign and the authors did not consider a realistic evaluation scenario in which new phishing emails are deployed.

Gualberto et al. (2020a) extracted distinctive features from the phishing emails' body text and performed several experiments using different number of features and ML algorithms and the best results accomplished by XGBoost algorithm (99.95% success rate). An extension of Gualberto et al. (2020a) presented in Gualberto et al. (2020b), where it was proposed a multi-stage solution for the detection of phishing emails that employs two methods (feature selection and feature extraction). The feature selection was based on Chi-Square statistics and Mutual Information, while feature extraction on Principal Component Analysis (PCA) and Latent Semantic Analysis (LSA). The best performance accomplished with LSA and the XGBoost algorithm. Both works deploy the same dataset to assess their approaches and the results were comparable; however, these works is that they did not take into account the phishing emails' growth, since the deployed phishing emails were outdated.

In Fang et al. (2019), the authors proposed THEMIS; a framework for the detection of phishing emails, which employs Recurrent Convolutional Neural Networks (RCNN). The emails' were divided into character and word levels and the Word2Vec method (Mikolov et al., 2013) was used to acquire the vector sequences from the aforementioned levels. THEMIS attained 99.848% detection accuracy with 0.043% false positive rate on the IWSPA 2018 dataset.[1] Towards the same direction, Hiransha et al. (2018) proposed a Deep Learning (DL) approach for the detection of phishing emails. The authors combined Convolutional Neural Networks along with Keras Word Embedding. The evaluation was performed on the IWSPA 2018 dataset using emails with and without headers and the results indicate that the model performs better when the email headers are not considered, accomplishing 96.8% accuracy. A common limitation of both Fang et al. (2019) and Hiransha et al. (2018) is that the authors did not test their models on newer emails to evaluate whether they can capture the evolution of phishing email attack. Moreover, in Hiransha et al. (2018) it was deployed an imbalanced ratio of samples and only the accuracy metric, which is inappropriate.

SAFE-PC (Gutierrez et al., 2018) is a methodology for detecting phishing emails based on text-based features, which were created using NLP techniques, such as Named Entity Recognition (NER), Freebase, and synonym substitution. The classification is performed via a RUSBoost algorithm, which integrates boosting and data sampling, with various ML algorithms and detected 71% of the phishing emails that had been erroneously classified by Sophos and achieved 15% FPR. Similarly, Egozi and Verma (2018) extracted in total 26 features based on word counts, punctuation, and stopwords, which were fed to various ML classifiers. The best performance achieved by SVM, which attained 83% TPR and 96% TNR. The main drawback of both Gutierrez et al.

[1] https://github.com/dasavisha/IWSPA-sharedtask

(2018) and Egozi and Verma (2018) is the weak feature sets that mainly relied on stop words, punctuation counts, variations of words, and different ratios among the features, which can be effortlessly evaded.

In Halgaš et al. (2020), the authors employed Recurrent Neural Networks (RNNs) to detect phishing emails utilizing only text-based features. The RNN classifier compared with a "textAnalysis" model (Verma et al., 2012) and a Dynamic Markov Chain (DMC) model (Bergholz et al., 2008), where it outperformed only (Verma et al., 2012). The limitations of this approach is that the authors did not provide sufficient information about the deployed phishing emails, therefore it is unknown whether this work can detect new phishing emails, as well as they did not base their evaluation on a real use case scenario using imbalance data.

Unnithan et al. (2018a) combined domain-level (i.e., the most commonly appeared words along with a list of special characters) and text-based features, which were extracted by the TF-IDF method. In the evaluation experiments the Logistic Regression classifier achieved the best F1-score (98%). The drawbacks of this work is that the employed dataset is obsolete and the domain-level features are susceptible, meaning that they can be easily evaded and reduce the efficacy of the approach. In another work, Unnithan et al. (2018b) compared the TF-IDF and Doc2Vec (a word embedding technique that is based on Word2Vec) methods for the extraction of text-based features from emails. Testing both methods on various ML algorithms the authors concluded that the Doc2Vec contributes to better detection performance. A major limitation of this work is that the highest accuracy that this approach reached (i.e., 88.4%) is not very good compared with the related works.

2.3. Ensemble learning classification

Ensemble Learning is a method where multiple ML algorithms are combined to solve a learning problem (e.g., classification). The combination of multiple algorithms, often, enhances the generalization ability of an Ensemble Learning model when compared with traditional ML models (Zhou, 2021). Namely, Ensemble Learning models can be used to advance the predictive performance of a classification task. In classification tasks, the individual algorithms of an Ensemble Learning model predict the class of a sample and a fusion method (e.g., Voting, Bagging, Stacking, etc.) is employed to combine the predictions of the individual algorithms concluding to a final prediction.

Ensemble Learning methods have been thoroughly examined in phishing website detection (Al-Sarem et al., 2021); however, in phishing email detection are still at its infancy. To the best of our knowledge, only a few works exist in the literature that investigate the efficacy of Ensemble Learning in the detection of phishing emails. The authors of Abawajy and Kelarev (2012) proposed an Ensemble Learning approach for phishing email classification that employs text-based features using TF-IDF method and syntactic features. Several Ensemble Learning methods were evaluated to identify the best options and the evaluation concluded that the combination of AdaBoost and Bagging methods improve the classification performance. Islam and Abawajy (2013) proposed a 3-tier phishing detection approach that deploys 21 hand-crafted features from the emails' contents. The authors tested various combinations of three classifiers and the arrangement AdaBoost (tier-1) - Naive Bayes (tier-2) - SVM (tier-3) outperformed all the others accomplishing 97% accuracy. The drawback of this work is that the extraction of hand-crafted features increases the system's complexity. In Yadav et al. (2017), the authors tested Ensemble Learning methods using content-based features and concluded that the stacking method outperforms weighted average and majority voting Ensemble Learning methods. The limitations of Abawajy and Kelarev (2012), Islam and Abawajy (2013), and Yadav et al. (2017) are further discussed in the next section.

2.4. Limitations of the literature

Table 1 presents a summary of the state-of-the-art ML approaches for phishing email detection that discussed in Section 2. The key aspects that are presented in Table 1 are the type of the feature set (i.e., Content-based or Text-based), the ML classifiers, as well as on the utilized benign and phishing email data sources. One can notice that only a few older existing works employ ensemble learning (e.g., Islam and Abawajy, 2013; Yadav et al., 2017; Abawajy and Kelarev, 2012) and that there is not an existing work that deploys a hybrid feature set that combines the content-based and text-based features.

Furthermore, although significant advances have been made in the past years in phishing email detection regarding the prediction performance there are still several limitations in the literature that have not been addressed by current detection solutions (Das et al., 2020). Regarding the existing works that deploy content-based features, the major drawbacks that were identified are: (i) outdated datasets (mainly due to the fact that newer methods did not consider content-based features), (ii) insufficient features, and (iii) poor evaluation experiments (e.g., limited evaluation metrics). Whilst, the major limitations of the existing works that focus on the textual features are (i) limited evaluation metrics and samples, (ii) limited experiments on newer phishing emails (i.e., the advancement of phishing emails is not taken into account), (iii) the generalization of the results on newer phishing emails has not been examined, and (iv) non-representative textual features. Another significant drawback of the literature is that the diversity of the data sources is not thoroughly considered, namely existing works did not use different datasets to perform the evaluation.

Ensemble Learning has accomplished very good results in other domains (e.g., Yerima and Sezer, 2018; Zhang et al., 2011); nevertheless, it has not been applied yet in the phishing email detection domain. Particularly, only a few existing works (Abawajy and Kelarev, 2012; Islam and Abawajy, 2013; Yadav et al., 2017) employ Ensemble Learning methods that suffer from several disadvantages. To name a few, the evaluation datasets comprised of a small number of old phishing emails, the deployed features are not enough to adequately represent the emails, and the evaluation metrics are insufficient (i.e., the AUC and accuracy were mostly used. Hence, there is a major gap regarding the applicability of Ensemble Learning in the detection of recent phishing email attacks.

To bridge the aforementioned gaps of the literature, we proposed HELPHED, a hybrid approach that combines the two most used feature categories of the literature (i.e., content-based and text-based features) to provide a comprehensive representation of the emails suitable for ML classification tasks. HELPHED is based on Ensemble Learning and employs two methods to process the hybrid features: Stacking Ensemble Learning (Method 1) and Soft Voting Ensemble Learning (Method 2). In this work, emphasis was put on the evaluation of the proposed methodology by proposing state-of-the-art guidelines (see Section 4.1 that contribute to the unbiased and accurate evaluation of HELPHED in order to provide a robust and effective methodology to curb the rising threat of email phishing attacks.

3. HELPHED

3.1. Overview

In this section, HELPHED is introduced, discussing its stages and describing the employed techniques in detail. HELPHED consists of 6 stages, as depicted in Fig. 1, whose main purpose switches between identifying representative features of phishing emails and parsing them to novel Ensemble ML algorithms in order later to assess their prediction performance on realistic data and real-life situations. Particularly, the proposed methodology starts with the *Email Parsing stage (S1)*, where the header and the body fields of an email are split and parsed to a vector structure as well as they are associated with the email's

Table 1
State-of-the-art ML approaches for phishing email detection.

Paper/Year	Content-based features	Text-based features	Classifiers	Benign dataset	Phishing dataset
Ma et al. (2009)	✓	✗	Decision Tree (C4.5)	N/A	N/A
Hamid and Abawajy (2011)	✓	✗	Bayesian Network	Anon. (2020f)	Anon. (2020e)
Moradpoor et al. (2017)	✓	✗	ANN	Anon. (2020f)	Anon. (2020e)
Akinyelu and Adewumi (2014)	✓	✗	Random Forest	N/A	N/A
Islam and Abawajy (2013)	✓	✗	Multi-tier Ensemble	Anon. (2020f)	Anon. (2020e)
Smadi et al. (2015)	✓	✗	Random Forest	Anon. (2020f)	Anon. (2020e)
Fette et al. (2007)	✓	✗	Random Forest	Anon. (2020f)	Anon. (2020e)
Chandrasekaran et al. (2006)	✓	✗	SVM	N/A	N/A
Abu-Nimeh et al. (2007)	✓	✗	Random Forest	N/A	Anon. (2020e)
Yadav et al. (2017)	✓	✗	Stacking Ensemble	Anon. (2020f)	Anon. (2020f)
Alhogail and Alsabih (2021)	✗	TF-IDF	GCN	Radev (2008)	Radev (2008)
Gualberto et al. (2020a)	✗	DTM & LDA	XGBoost	Anon. (2020f)	Anon. (2020e)
Gualberto et al. (2020b)	✗	TF-IDF & LSA	XGBoost	Anon. (2020f)	Anon. (2020e)
Fang et al. (2019)	✗	Word2Vec	RCNN	Anon. (2020c)	Verma et al. (2019)
Hiransha et al. (2018)	✗	Keras Word Emb.	CNN	Verma et al. (2019)	Verma et al. (2019)
Gutierrez et al. (2018)	✗	NER & Freebase	RUSBoost	N/A	N/A
Egozi and Verma (2018)	✗	Static features	Linear-SVM	Verma et al. (2019)	Verma et al. (2019)
Halgaš et al. (2020)	✗	Tokenized text	RNN	Anon. (2020f)	Anon. (2020e)
Unnithan et al. (2018a)	✗	TF-IDF & Domain Features	Logistic Regression	Verma et al. (2019)	Verma et al. (2019)
Unnithan et al. (2018b)	✗	TF-IDF & Doc2Vec	SVM	Verma et al. (2019)	Verma et al. (2019)
Abawajy and Kelarev (2012)	✗	TF-IDF & Syntactic	Multi-tier Ensemble	N/A	N/A

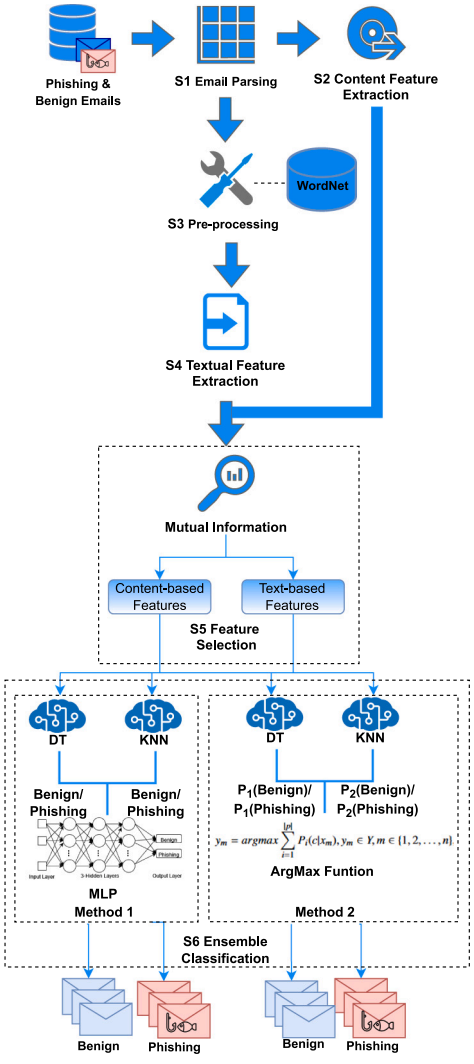


Fig. 1. HELPHED architecture.

respective class according to the collection from which it is originated (e.g., phishing or benign). Following, the *Content-based Feature Extraction stage (S2)* extracts the content-based features from the emails without further pre-processing. The content-based features include several parts of emails, such as *Body, Syntactic, Header, and URL features*. When the content-based features have been acquired, the *Pre-processing stage (S3)* takes place in which the body text of emails is converted into a uniform format. This stage includes a cleansing process step, where the texts are transformed into lowercase, the special characters, stopwords, and punctuation marks are deleted, and the hyperlinks are converted to a consistent string. Then, the lemmatization process is employed in the clean texts to consolidate the various inflected word forms and treat them as individual words. Later, the *Textual Feature Extraction stage (S4)* utilizes the Word2Vec method (Mikolov et al., 2013) to automatically extract the textual features. The innovation of the proposed work is that both content-based and text-based features were extracted from the emails and merged into a robust hybrid feature set to identify even the slightest variations between phishing and benign emails and hence offering a more advanced solution for the detection of the newer sophisticated phishing emails. The *Feature Selection (S5)* is the final stage prior to the classification, where the redundant features of both categories are identified by the *Mutual Information* method and excluded from the hybrid feature set. In this way, only the most informative features will be used in the classification stage. The *Ensemble Classification stage (S6)* of HELPHED deploys the selected hybrid features, which offer a comprehensive representation of emails to feed the ML algorithms for the detection of phishing emails by predicting the email's class (namely, phishing or benign). To provide a more comprehensive approach two methods have been considered. Particularly, Method 1 implements a *Stacking Ensemble Classifier*, which is comprised of 2-tiers; Tier-1 employs the DT and KNN ML algorithms to perform the initial classification and Tier-2 employs the MLP ANN algorithm, which is based on the classification results of Tier-1 to detect whether an email is phishing or benign. While Method 2 implements a *Soft-voting Ensemble Classifier* that employs in Tier-1 the DT and KNN algorithms for the initial classification of the emails and in Tier-2 a voting function to detect the phishing emails. Another innovation of this work is that using Ensemble Learning we accomplished the process of the hybrid features separately, namely using different ML algorithms to process the content-based and the text-based features. In this way, it can be deployed the ML algorithms that provide the best classification results when parsing each feature set.

The rest of this section is outlined as follows. In Section 3.2, it is portrayed how the data were converted from emails to a labeled dataset with respect to each email's class (i.e., benign or phishing). In Section 3.3 the content-based features are presented and the extraction process is described. In Section 3.4 it is explained the pre-processing stage, focusing on the text cleansing. In Section 3.5, the automatic extraction of text-based features is detailed, while Section 3.6 elaborates on the identification of the most informative features of each category (i.e., content-based and text-based). Finally, in Section 3.7, the ensemble methods for classification are made available, where *Method 1* is established on a *Stacking Ensemble* classification approach, while *Method 2* is based on the *Soft-voting Ensemble* classification approach.

3.2. Email parsing

In the Email Parsing stage (S1 of Fig. 1), the emails' header and message body fields are split and stored in separated arrays. This stage aims at the organization of emails in arrays to be processed faster in the next stages of HELPHED (i.e., S2–S6). The parsing operation springs from the fact that emails' format is intricate. Particularly, the email's body text contains tons of information that can facilitate researchers and security experts propose methods and tools for the detection of phishing emails. The main trait of body text is that it is arbitrary, because it is usually created by people. On the other hand, the phishing emails' body text disclose equivalent attributes, in deep semantics that differ from benign emails. Furthermore, phishing emails are usually comprised of multiple MIME parts (e.g., images and HTML code) that can be identified by the email's header fields, while benign emails mostly contain plain text.

All the emails undergo the parsing stage, wherein the corresponding body and header fields are extracted and stored in arrays B (Eq. (1)) and H (Eq. (2)) respectively, while the array L (Eq. (3)) stores the corresponding label of each email that represents the email's class. Where n is the number of samples (i.e., the number of both phishing and benign emails). The final step of the Email Parsing creates a joint array E (Eq. (4)) by merging the arrays B, H, and L. Array E is deployed by the content-based feature extraction stage, which extracts the content features from the body and header email fields, as well as by the pre-processing stage that prepares the emails' body texts for the text-based feature extraction stage.

$$B = \begin{bmatrix} b_1 & b_2 & b_3 & \dots & b_n \end{bmatrix} \tag{1}$$

$$H = \begin{bmatrix} h_1 & h_2 & h_3 & \dots & h_n \end{bmatrix} \tag{2}$$

$$L = \begin{bmatrix} l_1 & l_2 & l_3 & \dots & l_n \end{bmatrix} \tag{3}$$

$$E = \begin{bmatrix} b_{1\times1} & h_{1\times2} & l_{1\times3} \\ b_{2\times1} & h_{2\times2} & l_{2\times3} \\ b_{3\times1} & h_{3\times2} & l_{3\times3} \\ \vdots & \vdots & \\ b_{n\times1} & h_{n\times2} & l_{n\times3} \end{bmatrix} \tag{4}$$

3.3. Content-based feature extraction

In this section, it is presented the content-based feature extraction stage (S2 of Fig. 1). Before elaborating on the feature extraction procedure, the content-based features are analyzed, which are the first element of the proposed hybrid feature set. The deployment of the content-based features stems from the literature in the ML phishing email detection domain, where prior to the evolution of NLP methods, the existing works were heavily relied on the email's contents (i.e., features from specific parts of an email, such as headers, attachments, and suspicious words) to identify traits that accurately distinguish phishing from benign emails (Moradpoor et al., 2017). Thus, we argued that the contents of emails contain information that can be employed for phishing email detection. More specifically, the proposed methodology supports that phishing emails follow a particular format, where their contents differ from benign emails. Based on the format of emails as well as the phishing email traits, 4 feature categories have been identified:

- Phishing emails' are designed to perform particular malicious actions (e.g., deceiving the victim to share personal information or click a malicious attachment). To accomplish these actions, phishing emails employ *Body Features* that are related to the body structure of emails.
- The text of phishing emails is comprised of strong persuasion traits that are represented by *Syntactic Features* from the emails' body and subject fields.
- The emails include a header field that generates *Header Features*, which provide useful information about the email's type and the number of recipients.
- *URL Features* obtained from the hyperlinks and the domains of emails to indicate whether the email contains malicious hyperlinks or is generated by peculiar domains.

The content-based features are summarized in Table 2 and are further discussed in the following.

Body Features: The structure of phishing emails' body field differs from benign emails, mainly due to actions that an attacker does to deceive the victim to steal personal information or deliver malware. The *body features* category concentrates on traits that are located merely in the emails' body field and are depicted by binary and integer values. The binary values are represented by 0 or 1 and record whether the feature appears in an email, while integer values are represented by the number of appearances of a feature in an email.

1. **HTML Code:** This feature records the occurrence of HTML code in an email. (binary)
2. **HTML Forms:** This feature represents the presence of HTML-based forms in the email. HTML forms are frequently deployed by phishing campaigns to steal sensitive information. (binary)
3. **Scripts:** This feature tracks the presence of script code in the email (e.g. JavaScript). The scripts enhance the possibility for an email to include malicious code. The script codes are identified by the appearance of one of the following MIME types in the email: *text/javascript, application/javascript, text/ecmascript, application/ecmascript.* (binary)
4. **Attachments:** This feature represents the number of attachments in the email since phishing emails often deliver the malware via email attachments masqueraded as legitimate files (e.g., PDF). (integer)
5. **Image Link:** This feature records the presence of a hidden hyperlink behind an image. Phishing emails, frequently hide malicious hyperlinks (i.e., phishing URLs) behind images to draw the victim's attention and click the malicious link without their knowledge. (binary)

Syntactic Features: Overall, the syntax of benign emails is more abstract than phishing emails. While, the syntax of different phishing email campaigns presents similarities since their purpose is to persuade, frighten, or deceive the receiver to perform certain actions (e.g., click on a hyperlink, share her credential, etc.). The *syntactic features* category aims to the identification of various linguistic traits of phishing emails that vary from benign emails. These traits are portrayed by binary and numerical values.

6. **Bad words in email's body:** This feature counts the occurrences of certain words in the email's body. Various words appear more frequently in phishing emails than others. These words are: "link", "click", "confirm", "user", "customer", "client", "suspend", "restrict", "verify", "payment", and "protect". (integer)

Table 2
Content-based Features. The HELPHED value highlights the features that have been introduced by HELPHED.

Feature category	#	Feature name	Type	Description	Literature
Body features	**1**	**HTML Code**	**Binary**	**Presence of HTML code in email's body**	Akinyelu and Adewumi (2014), Smadi et al. (2015), Fette et al. (2007), Yadav et al. (2017)
	2	**HTML Forms**	**Binary**	**Presence of HTML Forms in email's body**	Ma et al. (2009), Smadi et al. (2015), Yadav et al. (2017)
	3	Scripts	Binary	Presence of scripting code in email's body	Ma et al. (2009), Moradpoor et al. (2017), Akinyelu and Adewumi (2014), Islam and Abawajy (2013), Fette et al. (2007), Yadav et al. (2017)
	4	**Attachments**	**Integer**	**No. of attachments in email**	Smadi et al. (2015)
	5	Image Link	Binary	Presence of a hidden hyperlink behind an image in email's body	Smadi et al. (2015)
Syntactic Features	**6**	**Bad words in email's body**	**Integer**	**No. of bad words that appear in email's body text**	Ma et al. (2009), Hamid and Abawajy (2011), Yadav et al. (2017)
	7	**Bad words in email's subject**	**Integer**	**No. of bad words that appear in email's subject**	Ma et al. (2009), Hamid and Abawajy (2011)
	8	**Absence of "RE" in subject**	**Binary**	**Presence of "RE" in email's subject**	**HELPHED**
	9	**No. of words in the email body**	**Integer**	**No. of words in email's body text**	Yadav et al. (2017), Toolan and Carthy (2009)
	10	**No. of characters in the email body**	**Integer**	**No. of characters in email's body text**	Yadav et al. (2017), Toolan and Carthy (2009)
	11	**Richness**	**Float**	**The ratio of the total number of words to the total number of characters**	Toolan and Carthy (2009)
	12	**Total number of distinct words in the email body**	**Integer**	**No. of distinct words in email's body text**	Yadav et al. (2017), Toolan and Carthy (2009)
	13	**Number of email parts**	**Integer**	**No. of different parts in an email**	Moradpoor et al. (2017), Smadi et al. (2015)
Header features	**14**	**Email encoding**	**Lexical**	**This lexical feature records the email's encoding type**	**HELPHED**
	15	**Number of email recipients**	**Integer**	**No. of recipients in an email**	**HELPHED**
URL features	**16**	**IP URL**	**Binary**	**Presence of IP instead of domain name in URL**	Hamid and Abawajy (2011), Akinyelu and Adewumi (2014), Smadi et al. (2015), Yadav et al. (2017)
	17	**Number of hyperlinks**	**Integer**	**No. of hyperlinks in an email**	Ma et al. (2009), Moradpoor et al. (2017), Akinyelu and Adewumi (2014), Smadi et al. (2015), Islam and Abawajy (2013), Fette et al. (2007)
	18	Text hyperlink	Binary	Presence of human readable text to hide a hyperlink	HELPHED
	19	**Number of different HREF**	**Integer**	**No. of different HREFs in an email**	Akinyelu and Adewumi (2014), Smadi et al. (2015), Fette et al. (2007)
	20	**Number of dots**	**Integer**	**Max No. of dots in URLs of an email**	Akinyelu and Adewumi (2014), Smadi et al. (2015), Fette et al. (2007), Chandrasekaran et al. (2006)
	21	Port URL	Binary	Presence of port in URLs of an email	Smadi et al. (2015), Yadav et al. (2017), Toolan and Carthy (2009)
	22	**Check domain**	**Binary**	**Presence of different sender's domain from hyperlink's domain**	**HELPHED**

7. **Bad words in email's subject:** Similar to the feature *Bad words in email's body* various words that appear in phishing emails subjects. These words are: "verify", "bank", "debit", "payment", and "suspend". This feature preserves the number of occurrences of the aforementioned words in the email's subject. (integer)
8. **Absence of "RE" in subject:** This feature tracks the presence of "RE" at the beginning of the email subject. The "RE" abbreviation is used in the subject of emails to indicate that it is a reply to another email. Usually, phishing emails do not have this abbreviation. (binary)
9. **No. of words in the email body:** This feature preserves the total number of words that are included in the email's body. (integer)
10. **No. of characters in the email body:** This feature counts the total number of characters in the email's body. (integer)
11. **Richness:** This feature records the ratio of the total number of words to the total number of characters. (float)

$$Richness = \frac{No.\ of\ words\ in\ the\ email\ body}{No.\ of\ characters\ in\ the\ email\ body}$$

12. **Total number of distinct words in the email body:** This feature counts the total number of distinct words that are included in the email's body. (integer)
13. **Number of email parts:** Multipart emails combine in the email body field different data types (e.g., images with HTML code). This integer feature includes the number of different parts in an email. (integer)

Header Features: The email headers when combined with features from the other categories can provide useful information for the detection of adversarial activity. This feature category is comprised of numerical and textual features.

14. **Email encoding:** This lexical feature records the email's encoding type using the Content-Transfer-Encoding header. The values of *Email encoding* feature are *Base64, Quoted-Printable, 8Bit, 7Bit, Binary, X-token*, and *None*. (textual)
15. **Number of email recipients:** This feature calculates the number of the email recipients. Often phishing campaigns simultaneously target multiple users. (integer)

URL Features: It is common for phishing emails to contain malicious URLs to be used as bait for the victims. To this end, a separate category of features that originated explicitly from URLs is considered. The *URL features* category is represented by binary and numerical values.

16. **IP URL:** This feature records the presence of a URL in an email that has an IP address instead of a domain name. It is common that temporary malicious URLs to be comprised of IP instead of domain names. (binary)
17. **Number of hyperlinks:** This feature calculates the number of hyperlinks that an email contains. (integer)
18. **Text hyperlink:** This feature represents the presence of human-readable text to hide a hyperlink. (binary)
19. **Number of different HREF:** This feature counts the cases in which the URLs of HREF HTML tags differ from the text that is appeared to the user. It has been noticed that phishing emails usually include hyperlink text which indicates a legitimate website, but instead they point to a phishing website. (integer)
20. **Number of dots:** This feature calculates the dots of each URL that appear in the email and records the highest number. It has been found that four dots or more is a usual indicator of a malicious URL (Jeeva and Rajsingh, 2016). (integer)
21. **Port URL:** This feature records the presence of URLs that contain a port number. Standard ports do not appear in the URLs and legitimate URLs rarely use non-standard ports. (binary)
22. **Check domain:** This feature tracks the presence of different sender's domain from the hyperlink's domain. (binary)

Please note that to the best of our knowledge, some of the features, such as *Absence of "RE" in the in subject, Email Encoding, Number of Email Recipients, Text Hyperlink, Check Domain*, have not been deployed by existing works that focused on the contents of emails; hence, the introduction of these features is part of the novelty of the proposed work.

The content-based feature extraction stage extracts the aforementioned features and their numerical values from the email's contents. Thus, we generate a joint set F that includes all the evident feature categories of the proposed feature set:

$$F = F_B \cup F_S \cup F_H \cup F_U \tag{5}$$

Where the *body, syntactic, header*, and *URL* feature categories are represented by F_B, F_S, F_H, and F_U respectively. More specifically, utilizing the set F, we define an $|F|$-dimensional vector space that includes binary values, real numbers, integers, and lexical values. To be more precise, 9 features are binary values, 11 features are integer numbers, 1 feature is a real number (float), and 1 feature is represented by a lexical value. The value of each feature is shown in Table 2. The sets F of all the emails are grouped together into a DataFrame (DF) as depicted in Array DF (Eq. (6)). DF is a matrix in which the rows represent the emails, while the first columns represent the extracted content-based features and the last one depicts the label of the email (namely, 0 for benign and 1 for phishing). The labels of the emails are already known since the emails have been acquired from public email datasets (see Section 4.2). At this point, we would like to clarify that the content-based features were extracted from the emails sequentially by a Python program, that receives as input the emails and generate the Array DF (Eq. (6)). Algorithm 1 represents the pseudocode of the content-based feature extraction stage. The input of the pseudocode (line 1) is Eq. (4) from the Email Parsing stage. Then, from each email (lines 2–12) the body, syntactic, header, and URL features are extracted (lines 4–8). Later, Eq. (6) is calculated based on the extracted features, which along with the label of the email (line 3) are used to construct Eq. (6) that is the output of the pseudocode (line 13).

$$DF = \begin{bmatrix} F_{B_1} & F_{S_1} & F_{H_1} & F_{U_1} & l_1 \\ F_{B_2} & F_{S_2} & F_{H_2} & F_{U_2} & l_2 \\ F_{B_3} & F_{S_3} & F_{H_3} & F_{U_3} & l_3 \\ \vdots & \vdots & & & \\ F_{B_n} & F_{S_n} & F_{H_n} & F_{U_n} & l_n \end{bmatrix} \tag{6}$$

Algorithm 1: Content-based Feature Extraction

```
1  Input E[] /* Equation (4)                                          */
2  for i in E do
3      label ← E.l[i];
4      F_Bi ← extract_body_features(E.B[i]);
5      F_Si ← extract_syntactic_features(E.B[i], E.H[i]);
6      F_Hi ← extract_body_features(E.B[i]);
7      F_Ui ← extract_url_features(E.B[i], E.H[i]);
8      F_i = F_Bi+F_Si+F_Hi+F_Ui;
       /* Equation (5)                                                */
9      DF.append(F_i);
10     DF.append(label);
11     i=i+1;
12 end
13 Output DF /* Equation (6)                                          */
```

3.4. Pre-processing

The pre-processing stage (S3 of Fig. 1) takes as input the emails' body text (i.e, the first column of E) and aims the elimination of redundant and trivial information. Furthermore, it contributes to the creation of a consistent format for all the emails' text. The pre-processing of the text is a common procedure that has been applied in many text classification tasks and can be found in several existing works (e.g., Gualberto et al., 2020b).

This stage aims to assist the *textual feature extraction* stage identifying the words of the email corpus that are more informative. The actions that pre-processing performs to accomplish its goals are depicted in Algorithm 2. The pseudocode has as input the body text of emails from Eq. (4) of the Email Parsing stage (line 1). Then the pre-processing begins by separating the words of the email's body text (lines 4–15) and converting the words into lowercase (5–7). Next, it removes all the stop words, special characters, punctuation marks, and HTML elements (lines 8–10). The hyperlinks that may appear in the email's body text are replaced with a fixed string (lines 11–13). The reason behind using a fixed string and not a common word, like "URL" or "LINK", is that the common word may appear in the text, hence the results will be falsified. Later, a tokenization process is performed (Alhogail and Alsabih, 2021), where a delimiter is used to split the words in order to convert the text into a list of words. In this work, the delimiter is a white space (e.g., space, new-line, tab) (line 16). By converting the text into a list, it is easier to process the words and perform the final step of the pre-processing stage, which is the reduction of a word's inflectional forms (line 17). Finally, the output is the pre-processed body text of the emails (line 18).

Lemmatization and stemming (Anandarajan et al., 2019) are the two most well-known processes for the reduction of a word's inflectional forms to convert it in its root form resulting in a smaller word set, since all the inflections of a word are converted to one (e.g., the words "better", "best", and "good" have the same root). Although they share the same purpose, stemming and lemmatization heavily differ. Stemming only trims the end or/and the beginning of a word based on prefixes and suffixes of inflected words. On the other hand, lemmatization is based on a thorough morphological analysis of the words and hence provides a meaningful root form of the words. To do

Algorithm 2: Email Body Text Pre-processing

```
Input E[];
/* Equation (4)                                                    */
e ← E.b[];
e ← e.split();
for w in e do
    if w is uppercase then
        convert w to lowercase;
    end
    if w=special_character OR w=stop_word OR w=HTML_element then
        remove w;
    end
    if w is hyperlink then
        w←"LINKLINK";
    end
    w_new ← w;
end
w_new ← w_new.split(" ");
w_new ← w_new.lemmatization();
Output Pre-processed Text;
```

so, the lemmatization process requires a vocabulary that contributes to the identification of the words' lemma (i.e., the word's original form). For the purposes of this work, the lemmatization method was deployed, since it provides a more accurate depiction of the words that appear in the emails and the WordNet database (Feinerer and Hornik, 2020) was used as a vocabulary. WordNet is a comprehensive lexical repository mainly of the English language that includes semantic relations between words, which also extends to more than 200 languages. In WordNet, the words are linked into semantic relations and grouped into cognitive synonym sets (synsets) that express the meaning of a notion. The interconnection among the synsets provides a mesh of substantially related words and notions that enhance the outcome of the NLP task.

In this work, WordNet aims to the reduction of the semantics of the emails' texts. Namely, via the lemmatization process, the diversity of the text is decreased by changing a group of words that have similar synonyms with their lemma. For example, the lemma of the word "sender" is the word "send", thus all the words "sender" in the email dataset are replaced with the word "send". Furthermore, the Part-of-Speech (POS) tagging method has been utilized to identify the grammatical category of each word (i.e, noun, verb, adjective, and adverb). POS helps the lemmatization process to further decrease the word instances. As one can understand, the lemmatization process significantly contributes to the dimensionality reduction of the emails' body text corpus, namely it facilitates the textual feature extraction stage to identify and extract more meaningful features.

3.5. Textual feature extraction

This section elaborates on the textual feature extraction stage (S4 of Fig. 1), providing details about the procedures that followed to result in the text-based feature set (i.e., the second element of the hybrid feature set). Latest works deal the phishing email detection as a text classification task exploiting the NLP advances in the phishing email detection domain and avoiding the manual extraction of features. To achieve the best classification results, comprehensive and adequate information about the data (i.e., emails) is needed. Thus, HELPHED considers the representation of emails as an integration of content-based and text-based features. Since emails' body text is controlled by people, it is arbitrary. On the contrary, phishing emails have a specific purpose, which is to obtain sensitive information from the victims, hence the body text message is often comprised of a warning, a threat or a fascinating content that aims to exploit people's psychological weaknesses and manipulate them to perform certain actions (e.g., click on a link). This renders the phishing emails' body text to be different in deep semantics than benign ones. Numerous NLP methods have been proposed in the literature to extract text-based features from phishing emails that are deployed by ML algorithms to distinguish benign and phishing emails, however, the two most used are the TF-IDF (Ramos et al., 2003) and Word2Vec (Mikolov et al., 2013).

Particularly, to effectively identify the deep semantics of emails, in this stage of HELPHED, the text-based features are extracted from the emails using the Word2Vec method. The rationale for selecting this method stems from our previous work (Bountakas et al., 2021), where it was performed a comparison of three well-known methods, namely TF-IDF, Word2Vec, and BERT (Devlin et al., 2018) in the detection of phishing emails using ML. The evaluation concluded that Word2Vec method is the most efficient choice for the detection of phishing emails (when focusing only on the textual information in the emails' body text). The output of the pre-processing stage is employed by the Word2Vec method that converts the email's body text into text-based features without manual interference. The rest of the subsection elaborates more on the Word2Vec method to assist the reader understand the *textual feature extraction* process.

Word2Vec (Mikolov et al., 2013) advances the text classification task and offers an individual perspective to the text mining domain due to its ability to convert words and phrases into vector representations. Word2Vec is a word embedding method that is able to capture the linguistic context of a word in a text, namely it identifies the relation of a target word with the surrounding words (e.g., the semantic and syntactic similarity). A two-layer ANN is employed to find the words' linguistic context, where the input is a large corpus of texts, which in this work is a large collection of emails and the generated outputs are vectors of particular dimensions. Specifically, Word2Vec creates a vectorized representation of words, named embeddings, in which each unique word in the collection has a corresponding vector linked with it and reflects the context of the word. Furthermore, similar words have similar vector representations, namely, they are close distance-wise in the embedded space. Word2Vec supports two techniques, (a) Continuous Bag of Words (CBOW), and (b) skip-gram. The CBOW technique predicts the target word based on the distributed representations of the context (e.g., surrounding words), while skip-gram technique parses the target word to identify the surrounding words. The CBOW method is faster than skip-gram, but skip-gram works better with infrequent words. Thus, in HELPHED, the skip-gram method were utilized together with the hierarchical softmax method for the training of the model. The train complexity of skip-gram is calculated by:

$$Q = C * (D + D * \log_2(V)), \tag{7}$$

where C is the maximum distance between the words, D is the word representation, and V is the dimensionality. The probability of correctly predicting a word w_i by giving a word w_j using the softmax method is depicted below. The two vectors u_w and v_w represent the word and the context respectively, since in skip-gram every word w is connected with the aforesaid vectors.

$$p(w_i|w_j) = \frac{e^{u_{w_i} v_{w_j}}}{\sum_{l=1}^{V} e^{u_l v_{w_j}}} \tag{8}$$

In total 300 text-based features have been extracted from the emails using the above mentioned Eqs. (7) and (8).

3.6. Feature selection

The feature selection stage (S5 of Fig. 1) is an essential data processing step that identifies the most informative features from the original feature set to enhance the performance of the ML classifiers in terms of accuracy and training time (Gnana et al., 2016). Its main objective is to remove irrelevant or redundant features without losing critical information. The feature selection process aids HELPHED to train the algorithms faster, avoiding overfitting, and obtaining better prediction results. Therefore, the features are ranked by a utility measure, through which those with the greatest values will be selected based on a threshold value. The remaining (unselected) features, namely those

with values less than the threshold value, are deleted and are not used by the ensemble classification stage.

Overall, the feature selection methods can be classified into four categories: filter, wrapper, embedded, and hybrid. The filter-based methods do not rely on a learning algorithm to identify the most informative features, instead, they evaluate the features according to heuristics based on general characteristics of the training data. On the other hand, the wrapper, embedded, and hybrid methods rely on a ML algorithm to identify the most informative features. More specifically, these methods conclude to the most informative features based on the algorithm's performance. Namely, they identify the most informative features specifically for the deployed algorithm. Hence, the latter methods contradict the generalization scope of the feature selection stage (i.e., to identify the most informative features for all the ML/DL algorithms). Moreover, another well-known method that has been used before for feature selection is PCA (Song et al., 2010). In order to understand why we did not deploy the PCA method, it should be clarified that the main goal of HELPHED's feature selection stage is to identify the most informative content-based and text-based features to enhance the training and classification performance of the Ensemble Learning methods. The PCA method is not a feature selection method per se, namely, it does not select the most informative features from a feature set; instead, it creates a new feature set with fewer features based on the original feature set (Alpaydin, 2020). Hence, in the literature is considered a feature extraction method. Existing works in the phishing email detection domain that deploy the PCA method, such as Gangavarapu et al. (2020) and Gualberto et al. (2020b); also support the argument that it is a feature extraction method rather than a feature selection method. Therefore, we believe that PCA method is inappropriate for HELPHED's feature selection stage. Based on the above observations, we deployed a filter-based method, named Mutual Information (MI).

MI is a well-known method for calculating the mutual dependence between two variables (or features), according to the entropy of a random variable. Particularly, MI calculates the quantity of information that is obtained in a random variable by observing another random variable, namely identifying the quantity of information each feature offers to determine the email's class. For two jointly continuous random variables, the MI is calculated by Eq. (9):

$$I(X,Y) = \int_y \int_x p_{(X,Y)}(x,y) \log \frac{p_{(X,Y)}(x,y)}{p_X(x) p_Y(y)} \, dx \, dy \tag{9}$$

where $I(X,Y)$ is a measure of dependency between the density of variable X and the density of the target Y, $p_X(x)$ and $p_Y(y)$ are the probability densities of x and y respectively, and $p_{(X,Y)}(x,y)$ is the probability of joint density (Guyon and Elisseeff, 2003). Finally, the input data in this task is the hybrid features, namely the content and text-based feature sets. Respectfully to the MI, we deployed Eq. (9) to calculate the value of each feature, and based on these values we identified the most informative features that will be used in the classification stage. The feature selection process is a challenging task due to the hybrid features. Thus, in order to obtain more robust features, we processed the content-based and text-based feature sets separately. Particularly, using Eq. (9) we estimated the MI score for each feature and classify them in descending order. The feature selection stage concluded in 18 content-based features (presented with bold font in Table 2) and in 253 text-based features (the evaluation of the features is presented in Section 4.4). Thus, the final hybrid feature set contains 271 features.

3.7. Ensemble classification

The Ensemble Classification stage (S6 of Fig. 1) is the nucleus of HELPHED. Since, the email samples (i.e., emails) are classified as phishing or benign, the Ensemble Classification stage is a binary classification task. The deployed email samples were obtained from publicly available sources (as explained in Section 4.2), viz the class of the emails is already known. Hence, the ensemble learning procedure of this stage is based on supervised learning.

The objective of the Ensemble Classification stage is to effectively process the hybrid features enhancing the classification performance of HELPHED. Towards this direction, instead of using a single classifier, HELPHED is based on Ensemble Learning. Ensemble Learning combines the results of multiple base learners (i.e., ML algorithms) to achieve more stable, robust, and accurate prediction performance than could be obtained by any of the base learners alone. In Ensemble Learning, the prediction results outputted by base learners are further processed (e.g., by a fusion model) to generate the final classification prediction. To augment the efficacy of HELPHED, two well-known Ensemble Learning methods were employed, named *Stacking Ensemble Learning (Method 1)* and *Soft Voting Ensemble Learning (Method 2)*. These methods were chosen because they can include different ML algorithm types, namely, they offer the flexibility to employ various ML algorithms to process the hybrid features separately (i.e., using an ML algorithm to process the content-based features and a different ML algorithm to process the text-based features). Method 1 and Method 2 differ in the way they fuse the results of the base learners and conclude to the final classification result.

An innovation of HELPHED is that the Ensemble Learning methods, which are employed to effectively process the hybrid features, are comprised of two base learners. More specifically, the hybrid feature set includes the content-based and text-based feature subsets. Each base learner processes these subsets separately, yet in parallel (e.g., the 1st base learner processes the content-based features, while the 2nd base learner processes the text-based features). The benefit of this approach is that it can deploy as base learners the ML algorithms that attain the best classification performance with each feature set.

Later, the results of the base learners are parsed by the fusion model to conclude to the final classification prediction. Moreover, the selection of the base learners that will be employed in Method 1 and Method 2 is a challenging task, since there are numerous ML algorithms appropriate for binary classification. To limit the available options HELPHED focused only on ML/DL algorithms that have been previously applied in phishing email detection (Das et al., 2020) accomplishing promising performance. Specifically, a pool of ML algorithms was created that included: (1) the function-based Logistic Regression (LR), (2) the probability-based Gaussian Naive Bayes (GNB), (3) the ANN Multilayer Perceptron (MLP), (4) the instance-based k-Nearest Neighbors (KNN), and (5) the tree-based Decision Tree (DT). Based on a thorough evaluation detailed in Section 4.5, we concluded that DT and KNN are the best base learners to process the content-based and text-based features respectively. The rest of the section elaborates on Stacking Ensemble Learning (Section 3.7.1) and Soft Voting Ensemble Learning (Section 3.7.2).

3.7.1. Method 1: Stacking ensemble learning

In HELPHED, the Stacking Ensemble Learning (Method 1) (Zhou, 2021) (depicted as *Method 1 - Stacking Classification* in Fig. 1) reinforces the detection of phishing emails by exploiting the predictions of two individual base learners, which process the hybrid features separately and in parallel, as well as by utilizing a fusion model (in Stacking Ensemble Learning the fusion is performed by an ML/DL algorithm) that combines these predictions to accurately classify the emails between phishing and benign. As depicted in Fig. 2, the Stacking Ensemble Learning model is comprised of 2-tiers. Tier-1 performs an initial classification of the emails, where the base learners classify them using the derived hybrid features, namely the content-based and the text-based feature sets. Particularly, the DT algorithm classifies an email based on the email's content-based features, while the KNN classifies an email based on its text-based features (Tier-1 of Fig. 2). The output of Tier-1 (i.e., the classification result of DT and KNN) is parsed in Tier-2, where a fusion model is applied. In Method 1, an ML algorithm is

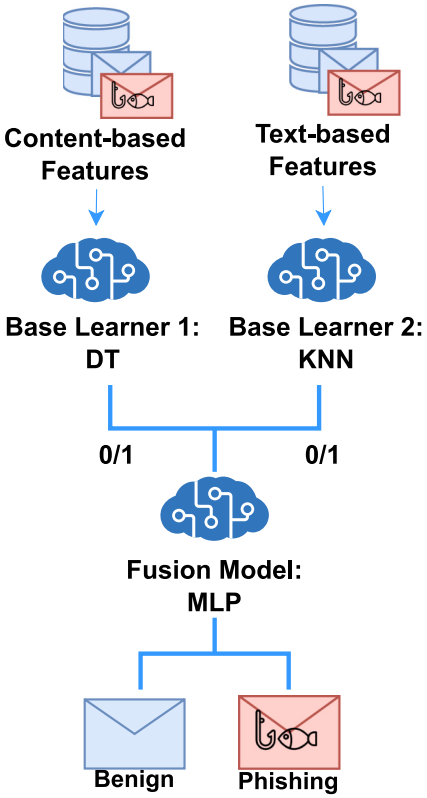


Fig. 2. Method 1 - Stacking Ensemble Learning.

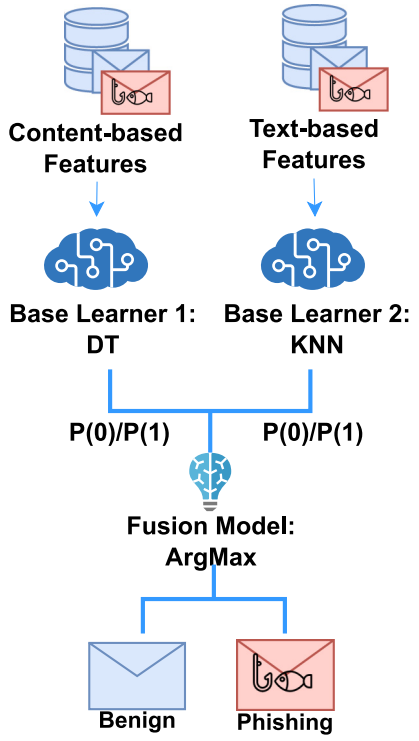


Fig. 3. Method 2 - Soft Voting Ensemble Learning.

applied as a fusion model to perform the final classification (Tier-2 of Fig. 2). Based on the evaluation explained in Section 4.5, the MLP DL algorithm was deployed as the fusion model since it can accomplish high detection performance with both the content-based and text-based features. Particularly, the MLP classifier parses the predictions of the DT and KNN base learners for each email, which is 1 for phishing and 0 for benign, and decides whether an email is phishing or benign.

3.7.2. Method 2: Soft voting ensemble learning

The Soft Voting Ensemble Learning (Method 2) (depicted as *Soft Voting Classification* in Fig. 1) considers the class prediction probabilities of each base learner and combines these predictions through an average function to perform the final classification. In general, there are two voting Ensemble Learning methods: (a) Majority Voting (Alotaibi and Alotaibi, 2021) and (b) Soft Voting (Dietterich, 2000), which differ on the way they process the results in the fusion model. Particularly, the Majority Voting takes the output of the base learners and performs the final classification based on the majority votes, while Soft Voting, averages the decisions of each base learner to perform the final classification. In this way, when compared with Majority Voting, the Soft Voting method eliminates the structure sensitivity that occurs from the base learners, reduces the ensemble's variance, and obtains more information.

As presented in Fig. 3, correspondingly to Method 1, two base learners are also employed in Method 2, namely the DT algorithm that processes the content-based features and the KNN algorithm that processes the text-based features (Tier-1 of Fig. 3). The output of both base learners is the probability that an email belongs to a certain class (e.g., P(phishing) or P(benign)). In Tier-2, these probabilities are used as input to an *Argmax* function to identify the most probable class and perform the final classification. More specifically, phishing email detection is a binary classification problem, where $Y = 0, 1$ (0 for benign and 1 for phishing) and a feature space $X \in \Re^k$, such that for any base learner F holds the function: $X \rightarrow Y$, then for each email sample (i.e., email) the decision profile of a base learner i is a pair of class probabilities $[P_{i_0}, P_{i_1}]$, where $\sum(P_{i_0}, P_{i_1}) = 1$. Consequently, the fusion model of the Soft Voting Ensemble Learning method, given an email sample X_m, combines the probabilities of the base learners to identify the most probable class (c) using Eq. (10).

$$y_m = argmax \sum_{i=1}^{|p|} P_i(c|x_m), y_m \in Y, m \in \{1, 2, \ldots, n\}, \tag{10}$$

where $|p|$ is the number of the base learners.

4. Experiments and results

In this section, it is presented a detailed evaluation of the proposed methodology focusing both on the deployed data and the prediction results. The experiments conducted in this paper were performed in a virtual machine located in a VMWare ESXi server with Intel Xeon 4114, 2.20 GHz x 4 CPU, and 12 GB RAM. The OS of the virtual machine was Ubuntu 20.04 64-bit. HELPHED was implemented using Python Scikit-learn (Pedregosa et al., 2011) and Apache Spark (Anon., 2020a). The Scikit-learn library was selected because it supports various state-of-the-art ML and DL algorithms that can be easily configured, giving us the opportunity to design appropriate models (e.g., by defining the values of hyperparameters) based on our needs (i.e., efficiently process the emails) and our goal (i.e., to enhance the phishing email detection performance). The evaluation approach along with the deployed evaluation guidelines that assist in avoiding biased results are described in Section 4.1. Section 4.2 elaborates on the datasets employed to evaluate HELPHED. In Section 4.3, the utility metrics to assess the outcomes of the proposed methodology are presented. Section 4.4 discusses the feature evaluation process, while Section 4.5 analyzes the procedure that is followed to select the best base learners for the ensemble classification stage comparing their performance on content-based and text-based features separately. The evaluation results of HELPHED are described in Section 4.6 and finally, Section 4.6.1 emphasizes on a

discussion between the proposed methodology and existing works. It should be noted that the results cannot be thoroughly explained, since the deployed hybrid features are not interpretable (because the text-based features are vector representations of the words that appear in the email); however, to facilitate the reader comprehend the efficacy of HELPHED, we will explicate the results based on the traits of the proposed methodology (e.g., ML/DL algorithms).

4.1. Evaluation approach

Several existing studies, such as Bountakas et al. (2022), El Aassal et al. (2020), Dou et al. (2017), and Quiring et al. (2022), have identified inaccuracies and pitfalls in the assessment of ML-based detection solutions, undermining the veracity of their obtained experimental results. These inaccuracies have been also identified in the phishing email detection domain (Das et al., 2020). Thus, to comprehensively and properly evaluate the detection capabilities of HELPHED, we proposed a set of state-of-the-art guidelines for the evaluation of ML algorithms in the phishing email detection field. Following, we highlight the most relevant guidelines that were taken into account to increase the robustness of the proposed methodology and avoid attaining misleading results.

- G1: Several researches (e.g., Egozi and Verma, 2018) share limited information regarding the datasets, the experimental environment, the features, and the ML algorithms, hence it is not possible to reproduce their implementation and repeat the experiments. Researchers should pay more attention to the reproducibility of their work.
- G2: The deployed phishing email samples should consider the evolution of phishing email attacks. Several recent works, such as Unnithan et al. (2018b), utilize old phishing emails (before 2015) that do not reflect the current attack trends. Violation of this guideline means that obsolete phishing emails are deployed and numerical results may be inflated.
- G3: The majority of existing works have been evaluated in balanced datasets or almost balanced (e.g., Moradpoor et al., 2017; Alhogail and Alsabih, 2021; Halgaš et al., 2020), namely the number of benign and phishing samples are almost the same. However, in real life, phishing email detection is an imbalanced classification problem, since the number of benign emails that an organization receives is much higher than phishing ones, hence in order to represent a realistic scenario the assessment should be performed on an imbalanced class ratio.
- G4: The ML-based detection method should be evaluated using appropriate evaluation metrics. Even though this may seem evident, there are plenty of works in the literature, such as Gualberto et al. (2020a) that use inappropriate evaluation metrics, namely they evaluate their method on an imbalanced class ratio by employing metrics that are influenced by class imbalance (e.g., detection accuracy).
- G5: A significant body of existing works did not employ diverse data sources, hence they lack generalization (e.g., Gualberto et al., 2020b). For instance, when using benign emails only from a specific organization the model learns to distinguish the particular email format that represents the organization's needs and if applied to another organization in a different field it might lead to more false positives.

The proposed work complies with all the above guidelines. More specifically, regarding ***Guideline G1***, the source code of HELPHED and the evaluation dataset are publicly shared to facilitate the research community to reproduce our results and compare HELPHED with feature solutions (Bountakas, 2021). The next sections are elaborate on the datasets and present how the assessment complies with the remaining guidelines (i.e., G2–G5).

4.2. Dataset

A cardinal objective of this work is to create a realistic dataset in order for the prediction results of HELPHED to be concrete. Consequently, several sources of publicly available raw data were deployed to enhance the diversity, such as the Enron Email Corpus (Anon., 2020c), the SpamAssassin Public Corpus (Anon., 2020f), the Nazario Phishing Corpus (Anon., 2020e), and authors' mailboxes (compliance with ***Guideline G5***). Before elaborating on each corpus it is important to mention that the IWSPA dataset (Verma et al., 2019), which is a well-known dataset among existing works, was not utilized because the dataset's creators have removed information that is important for the content-based features (e.g., headers, domains, and IP addresses), hence it could not be used in our experiments.

The Enron Corpus is a large dataset that is comprised of approximately 500,000 emails originated from 158 employees of the Enron Corporation. It is one of the few publicly available mass collections of real emails and it has been used in the past in various researches (Fang et al., 2019; Klimt and Yang, 2004; Kessler, 2010). The emails that correspond to interactions among different employees on day-to-day work issues, were publicly released during the legal investigation of the corporation. From the Enron Corpus, we randomly selected 28,000 emails to comprise our benign email samples, which to the best of our knowledge is the highest number among existing works. We deployed only a part of the Enron Corpus emails since in case the whole corpus was used, the benign samples of our dataset would not be diverse. Moreover, Spamassassin Corpus contains real email messages that originated from the Spamassassin developers and it was originally created for testing the Spamassassin filter. It contains in total 6047 emails of which 1897 are spam and the remaining 4150 are benign. The benign emails are further categorized on 3900 *easy_ham* (easy to differentiate them from spam emails), and 250 *hard_ham* (difficult to differentiate them from spam emails). In order to increase the variety of the benign samples, we chose to include all the benign emails from the Spamassassin Corpus (both *easy_ham* and *hard_ham*). After merging the Enron and Spamassassin Corpuses and dropping the duplicates the total benign email samples were 32,046.

The Nazario Phishing Corpus has been used as the source of raw phishing email data, which is also a public dataset that contains more than 4000 phishing emails approximately from 2003 to 2020 (at the time of writing) and it has been widely used to validate relevant research works. To consider the evolution of phishing emails, we deployed all the phishing emails from 2015–2020 (namely, 1472 emails) and 1992 emails that originated before 2015 in order to create a realistic scenario, where the phishing emails samples are approximately 10% of all the benign samples (the number of benign samples as mentioned above is 32,046). Furthermore, 76 phishing emails from the authors' mailboxes have been employed to enhance the diversity of the phishing samples as well as to include more recent phishing emails, since these emails have been received in the last couple of years (e.g., 2019–2021). The final phishing email dataset contains 3465 samples.

A common drawback of several existing works is that their assessment did not consider the evolution of phishing email attacks. Therefore, to abide by ***Guideline G2*** we have utilized 1548 phishing emails from the period 2015 to 2021. Moreover, to comply with ***Guideline G3***, an imbalanced class ratio has been deployed. Particularly, 1:10 ratio has been used between phishing and benign email classes (namely, the phishing emails comprise approximately 10% of the benign samples). The dataset was split into training and testing subsets, where 70% of the samples were used for training/validation and the remaining 30% used for testing the trained models, namely the training and testing sets are comprised of 24,857 (22,432 benign and 2425 phishing) and 10,654 (9619 benign and 1035 phishing) samples respectively. Table 3 depicts the number of email samples that constitute the training and testing sets as well as the total number of samples. Finally, to the best of our knowledge, the deployed dataset is the largest among recent

Table 3
The details of the evaluation dataset.

Dataset	Benign	Phishing	Total
Training set	22,432	2,425	24,857
Testing set	9,619	1,035	10,654
Total	32,051	3,460	35,511

existing works. The rationale for employing one large dataset instead of multiple smaller ones lies on the fact of representing a more realistic scenario (i.e., the emails originated from different organizations and the benign emails are much more than phishing). In this way, it is also addressed a limitation of the literature that identified by Das et al. (2020), which highlights the fact that existing solutions have been tested on smaller datasets that usually include samples originated from one source, hence their models might learn to distinguish the particular email format that represents the organization's needs and if applied to another organization in a different field it might lead to more false positives.

4.3. Metrics

In order to facilitate the reader to comprehend the performance of the proposed approach on real email data, we employed several evaluation metrics that offer a clear view of its prediction capabilities (compliance with ***Guideline G4***). The True Positive (TP) is the number of phishing emails that have been classified correctly and the True Negative (TN) is the number of benign emails that have been classified correctly. The False Positive (FP) is the number of benign emails that have been classified erroneously and the False Negative (FN) is the number of phishing emails that have been classified erroneously.

The evaluation of HELPHED relied in the following metrics:

- *Recall*: Illustrates the correctly classified phishing emails compared to the total number of phishing emails:

$$Recall = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{11}$$

- *Precision*: Represents the correctly classified phishing emails compared to number of emails that classified as phishing:

$$Precision = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{12}$$

- *False Positive Rate (FPR)*: Depicts the mistakenly classified benign emails compared to the total number of benign emails:

$$FPR = \frac{\text{FP}}{\text{FP} + \text{TN}} \tag{13}$$

- *False Negative Rate (FNR)*: Refers to the mistakenly classified phishing emails compared to the total number of phishing emails:

$$FNR = \frac{\text{FN}}{\text{TP} + \text{FN}} \tag{14}$$

- *Accuracy*: The percentage of correctly classified emails compared to the total number of emails:

$$Accuracy = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{15}$$

- *F1-Score*: A measurement to assess the prediction accuracy that considers both the *precision* and *recall*. The F1-score shows a better metric in case of imbalance datasets.

$$F1\text{-}Score = 2\frac{Precision * Recall}{Precision + Recall} = \frac{\text{TP}}{\text{TP} + \frac{1}{2}(\text{FP} + \text{FN})} \tag{16}$$

- *Matthews Correlation Coefficient (MCC)*: Is a measurement to assess binary classifications that is appropriate for imbalanced datasets, because it examines the accuracies and error rates on both classes (Bekkar et al., 2013). The higher the correlation between "true" and predicted values, the better the prediction.

$$\text{MCC} = \frac{\text{TP} * \text{TN} - \text{FP} * \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}} \tag{17}$$

- *Receiver Operating Characteristic (ROC) Curve*: Is a graph that plots the TPR and the FPR. More specifically, it illustrates the performance of a classification algorithm at different classification thresholds. Please note that the closer the ROC curve is to the shape of an upside down L, the better.
- *Area Under the ROC Curve (AUC)*: Measures the area under the ROC curve.
- *Training time*: The time that is required for a ML model to be trained is an important factor that most of the existing works neglect. Thus, we measured the training time of a model in seconds.

4.4. Feature evaluation

This section elaborates on the evaluation of the content-based and text-based features and describes how the final hybrid feature set was calculated using the MI feature selection method that described in Section 3.6. Fig. 4 depicts a diagram of the content-based features and their MI scores. The MI scores of the content-based features ranged between 0.20912 and 0.001813. In order to identify the most informative features for the detection of phishing emails, various experiments have been performed, where each time a different feature set was used based on the features' MI scores. More specifically, we picked: (i) features with above 0.1 MI score (4 features), (ii) features with above 0.05 MI score (11 features), (iii) features with above 0.01 MI score (18 features), and (iv) features with above 0.001 MI score (all 22 features). Overall, all the algorithms accomplished the best detection performance when selecting the feature set with above 0.01 MI score that included 18 features. Please note that the results of MLP were similar when employing MI scores of above 0.01 and 0.001, namely with 18 and 22 features; hence to achieve the best results with the least possible overhead, the feature selection stage led to 18 content-based features. Table 4 shows the best 5 content-based features based on their MI scores.

Regarding the text-based features, Fig. 5 illustrates a diagram of the text-based features and their MI scores. To identify the most informative features, we also utilized the same approach (i.e., we estimated the features' MI scores and classify them in descending order.) but employed more MI score thresholds to create more feature sets, since the text-based feature set (300 features) is significantly bigger than the content-based (22 features). Particularly, the MI scores of the text-based features ranged between 0.1439 and 0.00228. As previously mentioned, several experiments have been performed using different features each time. The deployed features were: (i) features with above 0.1 MI score (14 features), (ii) features with above 0.05 MI (93 features), (iii) features with above 0.02 MI score (190 features), (iv) features with above 0.015 MI score (217), (v) features with above 0.01 MI score (253), (vi) features with above 0.002286 MI score (300). The results when employing 190, 217, 253, and 300 features were close for all the ML algorithms, however, the best detection performance was attained when employing 253 features. Thus, the feature selection stage resulted in 253 text-based features. The 5 best text-based features are depicted in Table 5.

4.5. Selection of base learners

The selection of the two ML/DL algorithms (i.e., one algorithm will focus on the content-based features, while the other will focus on the text-based features) that will be deployed as base learners is a challenging task of the Ensemble Learning stage. This task is focused on the evaluation of the ML/DL algorithms on each feature set separately to identify the most efficient algorithm on each feature set. More

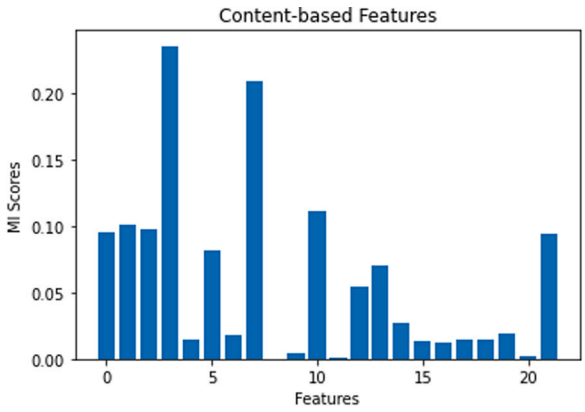


Fig. 4. Content-based features.

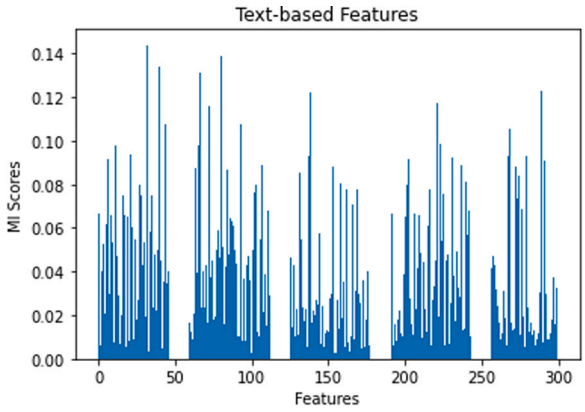


Fig. 5. Text-based features.

Table 4
Best Content-based features.

Feature	MI Score
HTML code	0.236191
Number of different HREF	0.209622
Number of dots	0.111287
Email Encoding	0.101805
Number of email parts	0.097852

Table 5
Best Text-based features.

Feature	MI Score
Feature 32	0.143940
Feature 80	0.139170
Feature 40	0.134125
Feature 250	0.133102
Feature 66	0.131291

specifically, we evaluated several well-known ML (LR, GNB, KNN, DT) algorithms and one DL algorithm (MLP) on content-based and text-based feature sets separately, using the datasets presented in Table 3, and the evaluation metrics discussed in Section 4.3. MLP was tested using various hidden layers (1, 2, 3, 4) and we concluded that it accomplishes the best performance with 3 hidden layers. Following, we demonstrate why HELPHED (both in Method 1 and Method 2) deploys the DT and KNN ML algorithms as base learners (as mentioned in Section 3.7) as well as why MLP was used as a fusion algorithm in Method 1 (see Section 3.7.1).

Regarding the results on the content-based features, the DT algorithm revealed the best classification performance on all the evaluation metrics (except training time), reaching 0.9885 F1-score, accuracy, precision, and recall, 0.9677 AUC, 0.9615 MCC outperforming all the other ML/DL algorithms, as presented in Table 6. More specifically, DT concluded to 9599 TNs, 42 FNs, 20 FPs, and 993 TPs. It is worth mentioning that the second-best performance achieved by the MLP algorithm (0.9806 F1-score, accuracy, and recall, 0.9809 precision, 0.9517 AUC, and 0.8909 MCC) as well as that the worst performance attained by the KNN algorithm (0.9387 F1-score, 0.9445 accuracy, 0.9406 precision, 0.9445 recall, 0.7589 AUC, and 0.6406 MCC). Based on the evaluation results on the content-based features, DT is the most appropriate algorithm for the classification of emails using only the content-based features. Thus, DT was deployed as the first base learner that processes the content-based features in HELPHED.

Regarding the results on the text-based feature set, KNN and MLP algorithms achieved comparable performance, as depicted in Table 7; however, the performance of KNN was slightly better. Particularly, KNN attained 0.9862 F1-score, accuracy, and recall, 0.9863 precision, 0.9635 AUC, 0.9218 MCC, as well as a very low training time (0.005 s) as a result of 9539 TNs, 67 FNs, 80 FPs, and 968 TPs. While, MLP accomplished 0.9812 F1-score, 0.9818 accuracy and recall, 0.9816 precision, 0.9179 AUC, and 0.8923 MCC which were the result of 9592 TNs, 167 FNs, 27 FPs, and 868 TPs. Contrariwise, the GNB algorithm revealed the worst performance (0.9369 F1-score, 0.932 accuracy and recall, 0.9466 precision, 0.8955 AUC, and 0.6831 MCC). Based on the evaluation results on the text-based features, KNN is the most appropriate algorithm for the classification of emails using only text-based features. Therefore, KNN was employed as the second base learner that processes the text-based features in HELPHED.

At this point, to encourage the reader to understand the presented notions, we provide the possible reasons (based on the traits of the deployed ML algorithms) that led to the best results. When it comes to the results using the content-based feature set, one possible reason that DT outperformed all the other algorithms is because there are features that are more informative than others and DT relies on them to perform the classification more than the other algorithms. Based on the features' evaluation (see Section 4.4), the 3 most informative features that facilitate DT reveal the best performance are the *HTML code, number of different HREF*, and *number of dots*. Considering the results using the text-based features, the potential reason that KNN attained better detection performance than the other algorithms is due to its ability to process a high number of features, namely the 252 text-based features, more effectively than the other algorithms. While MLP achieved comparable performance with KNN because a DL algorithm can deal both with a high number of samples and features more efficiently than the ML algorithms. Moreover, MLP accomplished very good performance (more than 98%) with both content-based and text-based features, thus it was deployed as a fusion algorithm in Method 1 (Section 3.7.1)

4.6. HELPHED evaluation

This section presents the evaluation of HELPHED (both with Method 1 and Method 2) on classifying the emails using the hybrid features. To highlight the effectiveness of HELPHED (i.e., the integration of Ensemble Learning with hybrid phishing email features), it was compared with traditional (state-of-the-art) ML and DL classifiers (i.e., LR, GNB, KNN, DT, and MLP). Furthermore, HELPHED was also compared with a prominent Ensemble Learning method, named Random Forest (RF) (Ho, 1995) to justify the advantage of the proposed methodology that processes the hybrid features separately (i.e., using an ML algorithm to process the content-based features and another ML algorithm to process the text-based features) against RF that processes the hybrid features like a unified feature set. The experiments were performed based on the evaluation guidelines described in Section 4.1 to avoid misleading or biased results. The dataset presented in Section 4.2 was deployed. The results revealed in this section were based on the metrics that

Table 6
Evaluation on content-based features.

Classifier	F1-score	Accuracy	Precision	Recall	AUC	MCC	Training time	TP	TN	FP	FN
LR	0.9756	0.9747	0.9778	0.9747	0.9674	0.8697	1.21	9392	992	43	227
GNB	0.972	0.9711	0.9737	0.9711	0.9499	0.8476	0.006	9390	956	79	229
KNN	0.9387	0.9445	0.9406	0.9445	0.7589	0.6406	**0.003**	9516	547	488	103
DT	**0.9885**	**0.9885**	**0.9885**	**0.9885**	**0.9677**	**0.9615**	0.016	**9599**	**993**	**42**	**20**
MLP	0.9806	0.9806	0.9809	0.9806	0.9517	0.8909	3.64	9499	948	87	120

Table 7
Evaluation on text-based features.

Classifier	F1-score	Accuracy	Precision	Recall	AUC	MCC	Training time	TP	TN	FP	FN
LR	0.9706	0.9692	0.9742	0.9692	0.9644	0.8147	64.467	9334	**992**	**43**	285
GNB	0.9369	0.932	0.9466	0.932	0.8955	0.6831	0.078	9049	880	155	570
KNN	**0.9862**	**0.9862**	**0.9863**	**0.9862**	**0.9635**	**0.9218**	**0.005**	9539	968	67	80
DT	0.9672	0.9671	0.9673	0.9671	0.9102	0.8163	0.284	9434	869	166	185
MLP	0.9812	0.9818	0.9816	0.9818	0.9179	0.8923	13.093	**9592**	868	167	**27**

were analyzed in Section 4.3, acquired from both email classes, namely phishing and benign, and their respective samples in the testing set (i.e., 9619 benign and 1035 phishing).

The results arranged in Table 8 present the performance of the ML classifiers as well as the performance of HELPHED with Method 1 and Method 2. As shown in Table 8, LR and GNB were poorly performed, hence we will not further discuss their results. Overall, the classification results of the remaining algorithms are enhanced when the hybrid features are employed (i.e., when considering both the content and text of emails). Particularly, in comparison to ML and ensemble RF classifiers, both HELPHED-Method 1 (Stacking) and HELPHED-Method 2 (Soft Voting) improved the overall detection performance. Method 2 attained better results than all the other classifiers in all the evaluation metrics (except training time). It misclassified only 2 benign emails as phishing (FPs) and 59 phishing emails as benign (FNs), hence it accomplished a high F1-score (0.9942), 0.9943 accuracy, precision, and recall, as well as 0.9714 AUC and 0.967 MCC. Moreover, it had a very low training time (0.0313 s) for an Ensemble Learning classification, which was the second-lowest training time after KNN (0.01 s). Following, Method 1 accomplished 0.9907 F1-score, accuracy, and recall, 0.906 precision, 0.969 AUC, and 0.9466 MCC which was the outcome of 39 FPs and 60 FNs.

Regarding the remaining classifiers, RF outperformed MLP, DT, and KNN. To be more precise, RF attained 0.9856 F1-score, 0.986 accuracy, 0.9805 precision, 0.9808 recall, 0.9323 AUC, and 0.918 MCC, which were the results of 10 FPs and 139 FNs. Thus, RF performed very well on the classification of benign emails and not well on the classification of phishing emails. MLP achieved 17 FPs and 143 FNs, which resulted in 0.9845 F1-score, 0.985 accuracy and recall, 0.9849 precision, 0.93 AUC, and 0.9117 MCC. DT had fewer FNs (120) than RF and MLP, but more FPs (85), hence it accomplished 0.9806 F1-score, 0.9808 accuracy and recall, 0.9805 precision, 0.9376 AUC, and 0.8887 MCC. KNN misclassified 102 FPs and 480 FNs, namely it accomplished 0.9398 F1-score, 0.9454 accuracy and recall, 0.9416 precision, 0.7628 AUC, and 0.6273 MCC. Finally, the ROC curves of all the classifiers are shown in Fig. 6. Please note that Ensemble Learning as well as the hybrid features are not interpretable, hence we cannot thoroughly analyze the experimental results and provide justifications about the reasons that led to the misclassifications. However, in the next section, we further discuss the evaluation results considering the ML algorithms' traits and provide answer to the RQ (*Can the combination of Ensemble Learning with hybrid features, overall, enhance the phishing email detection performance?*).

4.6.1. Discussions

Based on the tabulated results of Tables 6–8 we can answer the RQ. As one can notice by comparing the results on Tables 6 and 7 with Table 8, the only algorithm that obtained higher detection performance with the hybrid features is MLP. DT achieved slightly

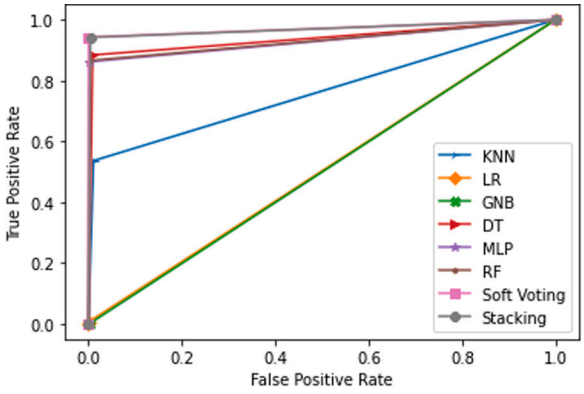


Fig. 6. Stacking classification approach.

worst performance, while the remaining algorithms (i.e., LR, GNB, KNN) attained worst performance. That is because MLP can handle data with very large dimensions (i.e., with a higher number of features) than the other algorithms, hence the hybrid features contribute to the enhancement of its performance. On the other hand, DT mostly focuses on the more informative features to perform the classification, which are the content-based features *HTML code, number of different HREF*, and *number of dots*, thus the rest of the content-based features, as well as the text-based features, did not influence its performance. LR, GNB, and KNN were not able to effectively process the high number of hybrid features, since they attained better results when processing the content-based or text-based features separately. More specifically, LR and GNB are not able to effectively process data with large dimensions (i.e., the text-based and hybrid features), while KNN could not effectively process the content-based features (see Table 6) affecting also on its performance when employing the hybrid features. Moreover, the rationale for employing the training time as a metric was the performance issues that may arise due to the combination of hybrid features with Ensemble Learning methods. Thus, we measured the performance of HELPHED using training time and as it is shown on Table 8 the training time of HELPHED with Method 2 is very low in comparison with the other ML/DL models (only the training time of KNN is lower) as well as the training time is similar to ML/DL models when they parse only the content-based (Table 6) or the text-based (Table 7) features. Therefore, we found that the hybrid features, overall, enhance the prediction results without significantly affecting the performance. Based on the above mentioned, regarding *RQ* it is evident that the combination of hybrid features with Ensemble Learning, overall, improves the phishing email detection performance (see Table 8), since both Method 1 and Method 2 outweighed all the other algorithms. Thus, we concluded that *Ensemble Learning when combined with hybrid features, overall, enhances the phishing email detection performance.*

Table 8
Classification results hybrid features.

Classifier	F1-score	Accuracy	Precision	Recall	AUC	MCC	Training time	TP	TN	FP	FN
LR	0.858	0.9028	0.8609	0.9028	0.503	0.0468	72.998	9611	7	1028	8
GNB	0.8561	0.9014	0.815	0.9014	0.4992	0.0123	0.095	9604	0	1035	15
KNN	0.9398	0.9454	0.9416	0.9454	0.7628	0.6273	**0.01**	9517	555	480	102
DT	0.9806	0.9808	0.9805	0.9808	0.9376	0.8887	0.407	9534	915	120	85
MLP	0.9845	0.985	0.9849	0.985	0.93	0.9117	17.005	9602	892	143	17
RF	0.9856	0.986	0.9805	0.9808	0.9323	0.918	4.038	9609	896	139	10
Method 1	0.9907	0.9907	0.9906	0.9907	0.969	0.9466	13.705	9580	975	60	39
Method 2	**0.9942**	**0.9943**	**0.9943**	**0.9943**	**0.9714**	**0.967**	0.0313	**9617**	**976**	**59**	**2**

4.6.2. Comparison of HELPHED's results with existing works

This section elaborates on the comparison of HELPHED's results with the results of prominent existing works from the period 2011–2022. Please consider that HELPHED cannot be compared directly with existing works due to the diversity of the evaluation approach that each work implements (i.e., different samples, class ratios, and metrics). Table 9 depicts the results of HELPHED and existing works in terms of the most common evaluation metrics (i.e., accuracy and F1-score), the category of the deployed features (i.e., content-based, text-based, and hybrid), as well as on the number of the utilized benign and phishing samples. Some existing works, such as Gualberto et al. (2020a) and Gualberto et al. (2020b) revealed very good detection performance; however, their results are not representative of how these works will carry out in real-life implementations since their assessment was performed on a significantly smaller dataset (4150 benign and 2279 phishing emails) that is exclusively comprised of obsolete phishing emails. Other works that accomplished good detection performance, like Fang et al. (2019) and Egozi and Verma (2018) apart from obsolete phishing emails, they employed inappropriate evaluation metrics to measure the prediction performance (i.e., the accuracy metric that is inappropriate for imbalanced data). Considering that HELPHED was evaluated using a large imbalanced dataset (32,051 benign and 3460 phishing emails) that considers the evolution of phishing emails and employs a realistic scenario where the phishing emails are much less than the benign, its numerical results with Soft-Voting (i.e., Method 2) are promising.

Moreover, the approach presented in Aljofey et al. (2022) also attained comparable performance on a large dataset. The authors of Aljofey et al. (2022) followed a similar approach with HELPHED to tackle phishing attacks but focused on the detection of phishing websites instead of phishing emails. More specifically, the similarities of Aljofey et al. (2022) and HELPHED are that both works deployed ML methods to perform the detection, employed hybrid features, and carried out a thorough evaluation using one large dataset instead of multiple smaller ones. Although there are a few similarities between the proposed work and Aljofey et al. (2022), there are more differences (except from the fact that they deal with different phishing domains). The major differences are that (a) HELPHED is based on Ensemble Learning to separately process the hybrid features and obtain higher detection performance and not on a boosting method as in Aljofey et al. (2022) (e.g., XGBoost); (b) the hybrid features of HELPHED are a combination of text-based and content-based features from emails, while the hybrid features of Aljofey et al. (2022) are the merge of URL and HTML features from websites; (c) HELPHED deployed Word2Vec to extract the textual information from emails instead of TF-IDF (Aljofey et al., 2022 uses TF-IDF to extract character level information).

To the best of our knowledge, HELPHED attained the highest scores that have been achieved in the phishing email detection domain using such a diverse dataset that considers the evolution of phishing emails. Furthermore, none of the existing works consider the guidelines proposed in Section 4.1 (i.e., G1–G5), hence their results might be misleading or biased (Das et al., 2020). Finally, a common drawback of existing works is that they did not share the appropriate information to reproduce their approach. To tackle this drawback, we have shared the utilized datasets and the code of HELPHED to drive more research in the phishing email detection domain and to facilitate researchers to reproduce the results of the proposed methodology and compare it with future solutions (Bountakas, 2021).

5. Conclusion

Phishing attacks occur frequently and are increasing in numbers by the day, especially in the current days with the remote working paradigm (due to COVID-19), where the security risks have been multiplied and the adversaries constantly seek new ways to exploit peoples' frustration (Karatisoglou et al., 2022). This research focused on the most common phishing attack paradigm, namely phishing emails. To this end, a hybrid feature set that thoroughly represents phishing emails was introduced, which considers both the contents and the textual information of emails. Furthermore, an Ensemble Learning approach that employs two methods (Stacking and Soft Voting) for the efficient detection of phishing emails was proposed. Both methods consist of two base learners to process the hybrid features separately, yet in parallel. Addressing the limitations of the literature, a diverse and comprehensive dataset was generated considering the evolution of phishing emails and the imbalance ratio between benign and phishing emails. Two exhaustive experiments were carried out to identify the best ML/DL algorithms to be deployed as base learners, as well as to extensively evaluate the proposed methodology with respect to state-of-the-art guidelines to avoid misleading and erroneous results. The results indicate that DT and KNN ML algorithms attain the best performance with the content-based and the text-based features respectively, hence these algorithms are deployed as base learners in the Ensemble Classification stage. Finally, the numerical results using the hybrid features revealed that HELPHED accomplishes the best performance with *Soft Voting Ensemble Learning (Method 2)* yielding 0.9942 F1-score, 0.9943 accuracy, precision, and recall, 0.967 MCC, as well as a low training time (0.0313 s) outperforming other Ensemble Learning methods and state-of-the-art ML/DL algorithms. In this way, we deduced the effectiveness of the proposed methodology in the detection of phishing emails, which is a result of the novelties of this work, namely the representation of emails using hybrid features and the deployment of Ensemble Learning classification methods that utilize two ML classifiers to processes the hybrid features. Based on the experimental results, we argue that the contribution of HELPHED is significant since the deployment of hybrid features provides a more thorough representation of emails and facilitates the Ensemble Learning methods to enhance the detection performance without affecting the training complexity (i.e., it has a very low training time).

As long as phishing emails are the initial step of large-scale attacks (e.g., APTs, Exploit Kits), they will remain a serious threat that constantly adapts to the current needs (as with COVID-19, where an outburst of phishing email attacks occurred). Since HELPHED provides a generic solution against phishing emails that considers both the contents and the textual information of emails, we strongly believe that it will be able to detect not only present but also future phishing emails. HELPHED can be easily combined with spam filters to shield corporate networks as well as individuals against phishing email attacks (Kalderemidis et al., 2022). An open problem in the phishing email detection domain is whether novel NLP models, which have been

Table 9
Results Comparison with State-of-the-art.

Paper/Year	F1-score (%)	Accuracy (%)	Feature category	# Benign samples	# Phishing samples
Hamid and Abawajy (2011)	–	92	Content	2364	2230
Moradpoor et al. (2017)	–	92.2	Content	6,656	7,714
Akinyelu and Adewumi (2014)	97.91	98.96	Content	1800	200
Islam and Abawajy (2013)	–	97	Content	N/A	N/A
Smadi et al. (2015)	98.09	98.11	Content	4,559	4,559
Alhogail and Alsabih (2021)	98.5	98.2	Text	4,894	3,685
Gualberto et al. (2020a)	99.9	99.9	Text	4,150	2,279
Gualberto et al. (2020b)	100	100	Text	4,150	2,279
Fang et al. (2019)	99.33	99.84	Text	7,781	999
Hiransha et al. (2018)	–	96.8	Text	5,088	612
Egozi and Verma (2018)	99	–	Text	7,689	1,210
Halgaš et al. (2020)	98.63	98.91	Text	6,951	4,572
Unnithan et al. (2018a)	98	97	Text	7,781	997
Unnithan et al. (2018b)	–	88.4	Text	8,913	1,087
Yadav et al. (2017)	–	98.02	Content	2,550	500
Aljofey et al. (2022)	96.38	98.28	Hybrid	32,972 (web-pages)	27,280 (web-pages)
HELPHED (Soft-voting)	99.41	99.42	Hybrid	32,051	3,460

proved powerful in text classification tasks, like BERT and GPT-3, are effective in phishing email classification tasks. Finally, we hope that HELPHED will drive more research towards hybrid feature schemes for the detection of phishing attacks, as well as that it will bring Ensemble Learning into the foreground of the phishing email detection domain.

CRediT authorship contribution statement

Panagiotis Bountakas: Conceptualization, Methodology, Investigation, Validation, Formal analysis, Data curation, Writing – original draft, Writing – review & editing, Visualization. **Christos Xenakis:** Methodology, Visualization, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors have shared the link with the data/code in the manuscript.

Acknowledgments

This research was partially funded by the European Commission (Horizon 2020 Programme), under the projects SECONDO (Grant Agreement no. 823997) and CyberSec4Europe (Grant Agreement no. 830929) as well as by the Greek State (Operational Programme Competitiveness, Entrepreneurship and Innovation 2014–2020 (EPAnEK)) under the project NetPHISH (Grant Agreement no. T1EΔK-05112).

References

Abawajy, J., Kelarev, A., 2012. A multi-tier ensemble construction of classifiers for phishing email detection and filtering. In: International Symposium on Cyberspace Safety and Security. Springer, pp. 48–56.

Abu-Nimeh, S., Nappa, D., Wang, X., Nair, S., 2007. A comparison of machine learning techniques for phishing detection. In: Proceedings of the Anti-Phishing Working Groups 2nd Annual ECrime Researchers Summit. pp. 60–69.

Akinyelu, A., Adewumi, A., 2014. Classification of phishing email using random forest machine learning technique. J. Appl. Math. 2014, http://dx.doi.org/10.1155/2014/425731.

Al-Sarem, M., Saeed, F., Al-Mekhlafi, Z.G., Mohammed, B.A., Al-Hadhrami, T., Al-shammari, M.T., Alreshidi, A., Alshammari, T.S., 2021. An optimized stacking ensemble model for phishing websites detection. Electronics 10 (11), URL https://www.mdpi.com/2079-9292/10/11/1285.

Alhogail, A., Alsabih, A., 2021. Applying machine learning and natural language processing to detect phishing email. Comput. Secur. 110, 102414.

Aljofey, A., Jiang, Q., Rasool, A., Chen, H., Liu, W., Qu, Q., Wang, Y., 2022. An effective detection approach for phishing websites using URL and HTML features. Sci. Rep. 12 (1), 1–19.

Alotaibi, B., Alotaibi, M., 2021. Consensus and majority vote feature selection methods and a detection technique for web phishing. J. Ambient Intell. Humaniz. Comput. 12 (1), 717–727.

Alpaydin, E., 2020. Introduction to Machine Learning. MIT Press.

Anandarajan, M., Hill, C., Nolan, T., 2019. Text preprocessing. In: Practical Text Analytics. Springer, pp. 45–59.

Anon., 2020a. Aparche spark - unified analytics engine for big data. https://spark.apache.org/. (Accessed November 2020).

Anon., 2020b. Enisa threat landscape 2020 - phishing. https://www.enisa.europa.eu/publications/phishing.

Anon., 2020c. Enron email dataset. http://www.cs.cmu.edu/~./enron/. (Accessed November 2020).

Anon., 2020d. Interpol COVID-19 cybercrime analysis report. https://www.interpol.int/News-and-Events/News/2020/INTERPOL-report-shows-alarming-rate-of-cyberattacks-during-{COVID}-19. (Accessed September 2020).

Anon., 2020e. Jose nazario phishing email corpus. https://monkey.org/~jose/phishing/. (Accessed November 2020).

Anon., 2020f. Spam assassin project (2015) spam assassin public corpus. https://spamassassin.apache.org/publiccorpus/. (Accessed November 2020).

Anon., 2021a. 2019 Phishing statistics and email fraud statistics. https://retruster.com/blog/2019-phishing-and-email-fraud-statistics.html. (Accessed December 2021).

Anon., 2021b. Phishing statistics report 2021. https://www.tessian.com/blog/phishing-statistics-2020/.

Anon., 2021c. Securing the enterprise in the COVID world, the state of email security. https://www.mimecast.com/state-of-email-security/.

Bekkar, M., Djemaa, H.K., Alitouche, T.A., 2013. Evaluation measures for models assessment over imbalanced data sets. J. Inf. Eng. Appl. 3 (10).

Bergholz, A., Reichartz, G.P.F., Strobel, S., Birlinghoven, S., 2008. Improved phishing detection using model-based features. In: In Fifth Conference on Email and Anti-Spam, CEAS.

Bhowan, U., Johnston, M., Zhang, M., Yao, X., 2013. Evolving diverse ensembles using genetic programming for classification with unbalanced data. IEEE Trans. Evol. Comput. 17 (3), 368–386. http://dx.doi.org/10.1109/TEVC.2012.2199119.

Bountakas, P., 2021. Helphed's data. https://drive.google.com/drive/folders/1zYKPbtifmqmGy8kjk4h9poIjvqlaA0fC?usp=sharing.

Bountakas, P., Koutroumpouchos, K., Xenakis, C., 2021. A comparison of natural language processing and machine learning methods for phishing email detection. In: The 16th International Conference on Availability, Reliability and Security. pp. 1–12.

Bountakas, P., Ntantogian, C., Xenakis, C., 2022. EKnad: Exploit Kits' network activity detection. Future Gener. Comput. Syst. 134, 219–235.

Chandrasekaran, M., Narayanan, K., Upadhyaya, S., 2006. Phishing email detection based on structural properties. In: NYS Cyber Security Conference, Vol. 3. Albany, New York.

Das, A., Baki, S., El Aassal, A., Verma, R., Dunbar, A., 2020. Sok: A comprehensive reexamination of phishing research from the security perspective. IEEE Commun. Surv. Tutor. 22 (1), 671–708. http://dx.doi.org/10.1109/COMST.2019.2957750.

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

Dietterich, T.G., 2000. Ensemble methods in machine learning. In: International Workshop on Multiple Classifier Systems. Springer, pp. 1–15.

Dou, Z., Khalil, I., Khreishah, A., Al-Fuqaha, A., Guizani, M., 2017. Systematization of knowledge (sok): A systematic review of software-based web phishing detection. IEEE Commun. Surv. Tutor. 19 (4), 2797–2819.

Egozi, G., Verma, R., 2018. Phishing email detection using robust NLP techniques. In: 2018 IEEE International Conference on Data Mining Workshops. ICDMW, pp. 7–12.

El Aassal, A., Baki, S., Das, A., Verma, R.M., 2020. An in-depth benchmarking and evaluation of phishing detection research for security needs. IEEE Access 8, 22170–22192.

Fang, Y., Zhang, C., Huang, C., Liu, L., Yang, Y., 2019. Phishing email detection using improved RCNN model with multilevel vectors and attention mechanism. IEEE Access 7, 56329–56340.

Feinerer, I., Hornik, K., 2020. Wordnet: WordNet interface. URL https://CRAN.R-project.org/package=wordnet. R package version 0.1-15.

Fette, I., Sadeh, N., Tomasic, A., 2007. Learning to detect phishing emails. In: Proceedings of the 16th International Conference on World Wide Web. pp. 649–656.

Gangavarapu, T., Jaidhar, C.D., Chanduka, B., 2020. Applicability of machine learning in spam and phishing email filtering: Review and approaches. Artif. Intell. Rev. 53 (7).

Gnana, D.A.A., Balamurugan, S.A.A., Leavline, E.J., 2016. Literature review on feature selection methods for high-dimensional data. Int. J. Comput. Appl. 975, 8887.

Gualberto, E.S., De Sousa, R.T., Thiago, P.D.B., Da Costa, J.P.C., Duque, C.G., 2020a. From feature engineering and topics models to enhanced prediction rates in phishing detection. IEEE Access 8, 76368–76385.

Gualberto, E.S., De Sousa, R.T., Vieira, T.P.D.B., Da Costa, J.P.C.L., Duque, C.G., 2020b. The answer is in the text: Multi-stage methods for phishing detection based on feature engineering. IEEE Access 8, 223529–223547.

Gutierrez, C.N., Kim, T., Corte, R.D., Avery, J., Goldwasser, D., Cinque, M., Bagchi, S., 2018. Learning from the ones that got away: Detecting new forms of phishing attacks. IEEE Trans. Dependable Secure Comput. 15 (6), 988–1001.

Guyon, I., Elisseeff, A., 2003. An introduction to variable and feature selection. J. Mach. Learn. Res. 3 (Mar), 1157–1182.

Haider, C.M.R., Iqbal, A., Rahman, A.H., Rahman, M.S., 2018. An ensemble learning based approach for impression fraud detection in mobile advertising. J. Netw. Comput. Appl. 112, 126–141.

Halgaš, L., Agrafiotis, I., Nurse, J., 2020. Catching the phish: Detecting phishing attacks using recurrent neural networks (RNNs). pp. 219–233.

Hamid, I.R.A., Abawajy, J., 2011. Hybrid feature selection for phishing email detection. In: International Conference on Algorithms and Architectures for Parallel Processing. Springer, pp. 266–275.

Hiransha, M., Unnithan, N., Vinayakumar, R., Soman, K., 2018. Deep learning based phishing E-mail detection CEN-deepspam.

Ho, T.K., 1995. Random decision forests. In: Proceedings of 3rd International Conference on Document Analysis and Recognition, Vol. 1, pp. 278–282.

Islam, R., Abawajy, J., 2013. A multi-tier phishing detection and filtering approach. J. Netw. Comput. Appl. 36 (1), 324–335.

Jeeva, S.C., Rajsingh, E.B., 2016. Intelligent phishing url detection using association rule mining. Hum.-centric Comput. Inf. Sci. 6 (10).

Kalderemidis, I., Farao, A., Bountakas, P., Panda, S., Xenakis, C., 2022. GTM: Game theoretic methodology for optimal cybersecurity defending strategies and investments. In: Proceedings of the 17th International Conference on Availability, Reliability and Security. pp. 1–9.

Karatisoglou, M., Farao, A., Bolgouras, V., Xenakis, C., 2022. BRIDGE: Bridging the gap between CTI production and consumption. In: 2022 14th International Conference on Communications. COMM, IEEE, pp. 1–6.

Kessler, G., 2010. Virtual business: An enron email corpus study. J. Pragmat. 42 (1), 262–270.

Klimt, B., Yang, Y., 2004. The enron corpus: A new dataset for email classification research. In: European Conference on Machine Learning. Springer, pp. 217–226.

Li, Y., Yang, Z., Chen, X., Yuan, H., Liu, W., 2019. A stacking model using URL and HTML features for phishing webpage detection. Future Gener. Comput. Syst. 94, 27–39.

Ma, L., Ofoghi, B., Watters, P., Brown, S., 2009. Detecting phishing emails using hybrid features. In: 2009 Symposia and Workshops on Ubiquitous, Autonomic and Trusted Computing. pp. 493–497.

Marchal, S., François, J., State, R., Engel, T., 2014. PhishStorm: Detecting phishing with streaming analytics. IEEE Trans. Netw. Serv. Manag. 11 (4), 458–471.

Mikolov, T., Chen, K., Corrado, G., Dean, J., 2013. Efficient estimation of word representations in vector space. arXiv:1301.3781.

Moradpoor, N., Clavie, B., Buchanan, B., 2017. Employing machine learning techniques for detection and classification of phishing emails. In: 2017 Computing Conference. pp. 149–156.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E., 2011. Scikit-learn: Machine learning in Python. J. Mach. Learn. Res. 12, 2825–2830.

Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K., 2022. Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium, USENIX Security 22. USENIX Association, Boston, MA.

Radev, D., 2008. Clair collection of fraud email, acl data and code repository. ADCR2008T001.

Ramos, J., et al., 2003. Using tf-idf to determine word relevance in document queries. In: Proceedings of the First Instructional Conference on Machine Learning, Vol. 242. New Jersey, USA, pp. 133–142.

Smadi, S., Aslam, N., Zhang, L., Alasem, R., Hossain, M.A., 2015. Detection of phishing emails using data mining algorithms. In: 2015 9th International Conference on Software, Knowledge, Information Management and Applications. SKIMA, pp. 1–8.

Song, F., Guo, Z., Mei, D., 2010. Feature selection using principal component analysis. In: 2010 International Conference on System Science, Engineering Design and Manufacturing Informatization, Vol. 1. IEEE, pp. 27–30.

Toolan, F., Carthy, J., 2009. Phishing detection using classifier ensembles. In: 2009 ECrime Researchers Summit. IEEE, pp. 1–9.

Unnithan, N.A., Harikrishnan, N., Akarsh, S., Vinayakumar, R., Soman, K., 2018a. Machine learning based phishing e-mail detection. pp. 65–69, Security-CEN@ Amrita.

Unnithan, N.A., Harikrishnan, N., Vinayakumar, R., Soman, K., Sundarakrishna, S., 2018b. Detecting phishing E-mail using machine learning techniques. In: Proc. 1st Anti-Phishing Shared Task Pilot 4th ACM IWSPA Co-Located 8th ACM Conf. Data Appl. Secur. Privacy. CODASPY, pp. 51–54.

Verma, R., Shashidhar, N., Hossain, N., 2012. Detecting phishing emails the natural language way. In: Foresti, S., Yung, M., Martinelli, F. (Eds.), Computer Security – ESORICS 2012. pp. 824–841.

Verma, R.M., Zeng, V., Faridi, H., 2019. Data quality for security challenges: Case studies of phishing, malware and intrusion detection datasets. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. CCS '19, Association for Computing Machinery, New York, NY, USA, pp. 2605–2607. http://dx.doi.org/10.1145/3319535.3363267.

Yadav, D.P., Paliwal, P., Kumar, D., Tripathi, R., 2017. A novel ensemble based identification of phishing e-mails. In: Proceedings of the 9th International Conference on Machine Learning and Computing. pp. 447–451.

Yamin, M.M., Ullah, M., Ullah, H., Katt, B., 2021. Weaponized AI for cyber attacks. J. Inform. Secur. Appl. 57, 102722.

Yerima, S.Y., Sezer, S., 2018. Droidfusion: A novel multilevel classifier fusion approach for android malware detection. IEEE Trans. Cybern. 49 (2), 453–466.

Zhang, H., Liu, G., Chow, T.W.S., Liu, W., 2011. Textual and visual content-based anti-phishing: A Bayesian approach. IEEE Trans. Neural Netw. 22 (10), 1532–1546.

Zhou, Z.-H., 2021. Ensemble learning. In: Machine Learning. Springer, pp. 181–210.

Mr. Panagiotis Bountakas holds a B.Sc. in Information and Communication Technologies (2015) and a M.Sc. in Digital Systems Security (2018) both from the *Digital Systems Department of University of Piraeus*. Currently, he is a Ph.D. student in University of Piraeus and a research assistant in Systems Security Laboratory. His research interests focus on data collection, analysis of cyber-attacks and malware detection using machine and deep learning methods. The last years he has participated in several EU H2020 Programs, as well as in Greek state projects and his research has been published in renowned journals and conferences. He is an active reviewer in numerous Information Security journals and conferences.

Professor Christos Xenakis received his B.Sc. degree in computer science in 1993 and his M.Sc. degree in telecommunication and computer networks in 1996, both from the Department of Informatics and Telecommunications, University of Athens, Greece. In 2004 he received his Ph.D. from the University of Athens (Department of Informatics and Telecommunications). Since 2007 he is a faculty member of the Department of Digital Systems of the University of Piraeus, Greece, where currently is a Professor, a member of the Systems Security Laboratory, the director of the Postgraduate Degree Programme on "Digital Systems Security", a steering committee member of the European Cyber Security Challenge (ECSC), and the leader of the Hellenic Cyber Security Team. He has participated in numerous projects realized in the context of EU Programs. He is the project manager of the CUREX, SECONDO, INCOGNITO and SealedGRID projects, funded by Horizon2020. He is a member of the editorial board of three Thomson Reuters indexed journals.